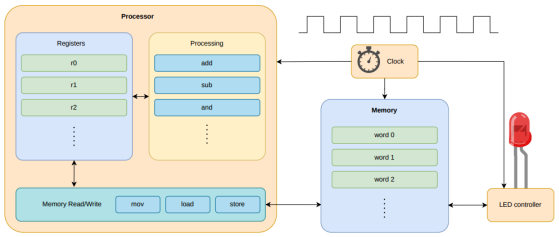
Curs	Topic	Status
UART & SPI	dma uart spi sensors	c'est fini
Introduction into PM		ițdan
GPIO		gone, reduced to atoms
Async		it's over, isn't it?
I2C & USB		Blasting off again
Certificari		Adios
WiFI		
ZigBee		Until we meet again
PM_Power		

Introduction to PM

	Microcontroller(MCU)	Microprocesor(CPU)
Operating Frequency	Low(MHz)	High(GHz)
I/O	Lots of ports	Limited number
Operating System	Doesn't require	Requires
Cost	0.1-25$	75-500$
Demand	Bilions	Tens of Millions
Function	CPU, mem, I/O	CPI
Complexity	simple	complex
USe case	Incorporated devices	PC, Sever, Laptop
	controls hardware	



von Neumann VS Harvard

- von Neumann, where memory contains both instructions and data.
- Today's PCs are all von Neumann Harvard, where memory access is done on separate buses, one for data, one for instructions.
- AVR,PIC, DSPs and many microcontrollers are Harvard
- Note: ARM is von Neumann with some * Note: GPUs (NVIDIA) are mixed arhitecture

	RP2350	Atmega328P
Cores	2	1
Word	32 bit	8 bit
Freq	up to 150MHz	up to 16MHz
RAM	520 KB	32KB Flash & 1 KB EEPROM

RP2350 Memory Breakdown

Memory Type	Size	Purpose
XIP [1] Flash	Up to 16 MB	Stores program code (external QSPI Flash).
SRAM (On-chip)	520 KB	Stores stack, heap, variables, and data.
Boot ROM	32 KB	Stores bootloader, factory firmware.
OTP	8 KB	One-time-programmable (Product id, cryptographic keys).
Peripheral Space	Varies	Memory-mapped I/O for GPIO, UART, SPI, DMA.
Registers	16 + control registers	General purpose + program flow + special purpose

1. XIP = Execute in Place (without this, the code would need to be copied in RAM first) ↩

ATmega328P Memory Details

Memory Type	Size	Purpose
Flash (ROM)	32 KB	Stores program instructions (non-volatile).
SRAM (RAM)	2 KB	Stores variables, stack, heap, and registers.
EEPROM	1 KB	Stores persistent data (non-volatile, writable).
General Purpose Registers	32 Bytes	Fast-access CPU registers.
I/O Registers	64 Bytes	Port-mapped peripheral control registers.
Extended I/O Registers	160 Bytes	Memory mapped peripheral control registers.

Rust

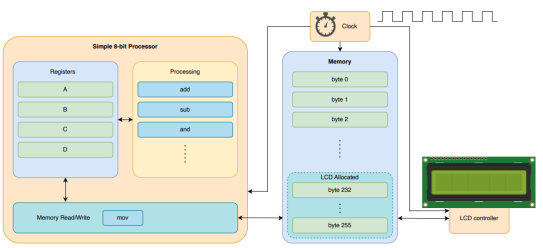
The tagline of Rust is No Unintended Behavior.

- no null reference; the Rust compiler explicitly asks developers to check this;
- no implicit cast, even adding a u32 to a u8 must be casted;
- safe access to shared data across threads veried at compile time;
- uses type states to move runtime checks to compile time and force developers to check;
- clearly defined data types, unlike i8 or u128;
- safe unions,that provide a discriminant to prevent wrong interpretation of data;
- clear code organization into crates and modules;
- backward compatibility at crate level

Operatii pe Biți

Set bit:
register | 1 << bit
register | mask (pt mai multi biti)
Clear bit
register & !(1 << bit)
register & !mask
Flip bit
register ^ 1 << bit
register ^ mask
Extragem din 32 biti, primii 12 cei mai semnificativi
const MASK: u32 = 0b0000_0000_0000_0000_0000_1111_1111_1111;
let extracted_bits = (large_id >> 20) & MASK;

MMIO(Memory mapped input output)




```
for character in b"Hello, World".iter() {
  // character is &char, so we use *character to get the value
  unsafe { UART_TX.write(*character); }
}
```

Optimizat in:

```
const UART_TX: *mut u8 = 0x400_3400;
unsafe { UART_TX.write(b'd'); }
```

Folosim **write/read_volatile**, pentru a nu optimiza compilatorul:

```
for character in b"Hello, World".iter() {
  // character is &char, so we use *character to get the value
  unsafe { write_volatile(UART_TX, *character); }
}
```

Cache

- Cache types:
  - write-through - data is written to the cache and to the main memory (bus)
  - write-back - data is written to the cache and later to the main memory (bus)
- few Cortex-M MCUs have cache
- the Memory Mapped I/O region is set as nocache
- for chips that use cache
  - nocache regions have to be set manually (if MCU knows)
  - or, the cache has to be flushed before a volatile_read and after a volatile_write
  - beware DMA controllers that can't see the cache contents

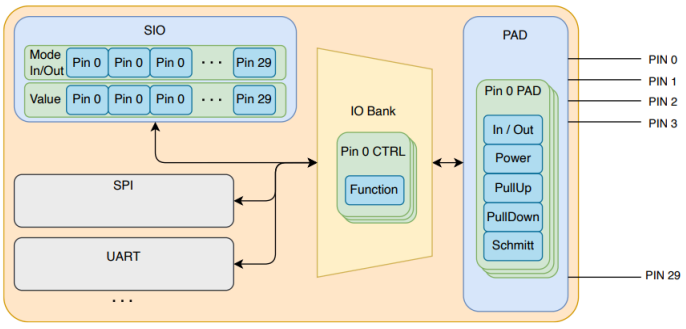
GPIO on AVR

DDRx - Data Direction Register. If the data direction bit is 1, the pin is an input. 0 is an output.
1 DDRB &= ~(1<<DDB7); // Makes pin PB7 an input
2 DDRB |= (1<<DDB5); // Makes pin PB5 an output

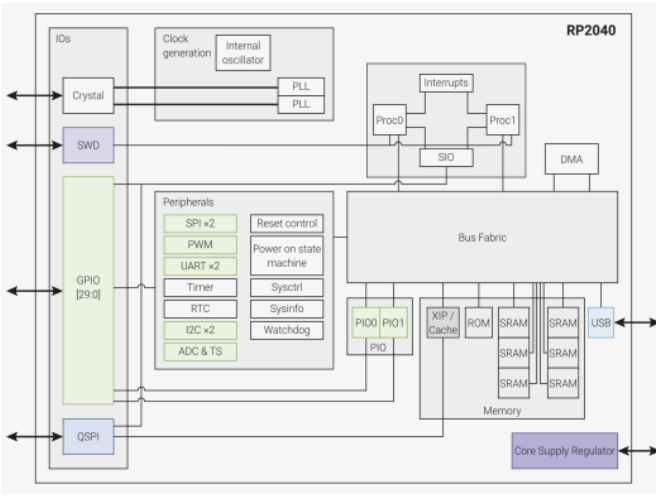
PORTx - Port Data Register. If a pin is configured as an output, setting the corresponding bit to 1 makes the pin output high. 0 makes the output low.
1 PORTB |= (1<<PORTB5); // makes pin B5 high (e.g.: a LED would turn on)

PINx - Port Pin Register. Used to read the pin state.
1 if (PINB & (1<<PINB7) == 1) { ... } // pin B7 reads a high value

GPIO(General Purpose Input Output)



Peripherals:
SIO Single Cycle Input/Output, is able to control the GPIO pins
GPIO Multiplexes the functions of the GPIO pins
SIO: Set the pin as Input or Output
IO Bank (GPIO): Use the correct MUX function (F5)
PAD: Set the pin input and output parameters



Offset	Name	Info
0x000	CPUID	Processor core identifier
0x004	GPIO_IN	Input value for GPIO pins
0x008	GPIO_HI_IN	Input value for QSPI pins
0x010	GPIO_OUT	GPIO output value
0x014	GPIO_OUT_SET	GPIO output value set
0x018	GPIO_OUT_CLR	GPIO output value clear
0x01c	GPIO_OUT_XOR	GPIO output value XOR
0x020	GPIO_OE	GPIO output enable
0x024	GPIO_OE_SET	GPIO output enable set
0x028	GPIO_OE_CLR	GPIO output enable clear

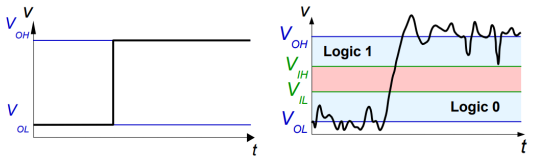
- Input
  - set GPIO_OE bit x to 0
  - read GPIO_IN bit x
- Ouput
  - set GPIO_OE bit x to 1
  - write GPIO_OUT bit x

Citire:
const GPIO_IN: *const u32= 0xd000_0004 as *const u32;
let value = unsafe {read_volatile(GPIO_IN) >> pin & 0b1 };
Scriere:
const GPIO_OE_SET: *mut u32= 0xd000_0024 as *mut u32;
const GPIO_OUT: *mut u32 = 0xd000_0010 as *mut u32;
unsafe {write_volatile(GPIO_OE_SET, 1 << pin);
write_volatile(GPIO_OUT, gpio_out); }

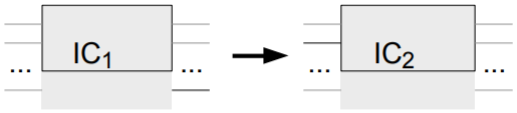
Bits	Name	Description	Type	Reset
31:30	Reserved.	-	-	-
29:28	IRQOVER	0x0 → don't invert the interrupt 0x1 → invert the interrupt 0x2 → drive interrupt low 0x3 → drive interrupt high	RW	0x0
27:18	Reserved.	-	-	-
17:16	INOVER	0x0 → don't invert the peri input 0x1 → invert the peri input 0x2 → drive peri input low 0x3 → drive peri input high	RW	0x0
15:14	Reserved.	-	-	-
13:12	OEOVER	0x0 → drive output enable from peripheral signal selected by funcsel 0x1 → drive output enable from inverse of peripheral signal selected by funcsel 0x2 → disable output 0x3 → enable output	RW	0x0
11:10	Reserved.	-	-	-
9:8	OUTOVER	0x0 → drive output from peripheral signal selected by funcsel 0x1 → drive output from inverse of peripheral signal selected by funcsel 0x2 → drive output low 0x3 → drive output high	RW	0x0
7:5	Reserved.	-	-	-
4:0	FUNCSEL	Function select. 31 == NULL. See GPIO function table for available functions.	RW	0x1f

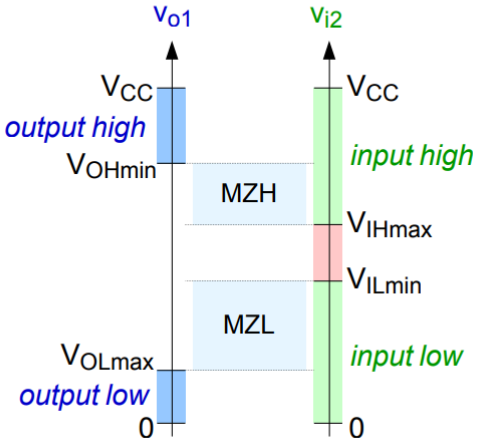
Signals

-analog signals are real signals
-digital signals are a numerical representation of an analog signal (software level)
-hardware usually works with two-level digital signals (hardware level)

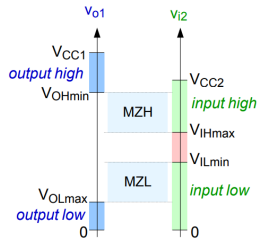
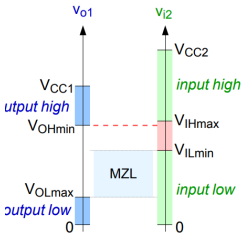


Signals between VIL and VIH risk to be misinterpreted



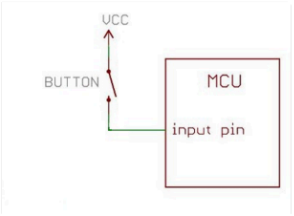


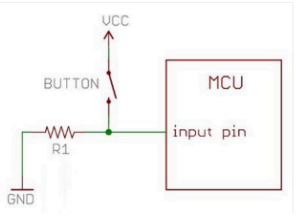
Conditions:
VOH_transmiter > VIH_receiver
VOL_transmiter < VIL_receiver

VCC1 > VCC2		Might work, might produce magic smock Problem: VOH_transmiter > V CCreceiver (the high won't be interpreted as high by device 2)
VCC1 < VCC2		VCC_transmiter ≤ VIH_receiver might not work

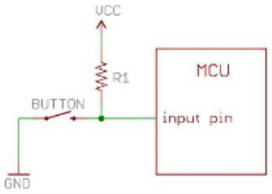
Pull up/down resistors

Pull down R





First: when the button is not pressed, the signal could be low or high
Second: when the button is not pressed, the signal is low
Pull Up R
When the button is pressed, it sends a low signal.



Interrupts

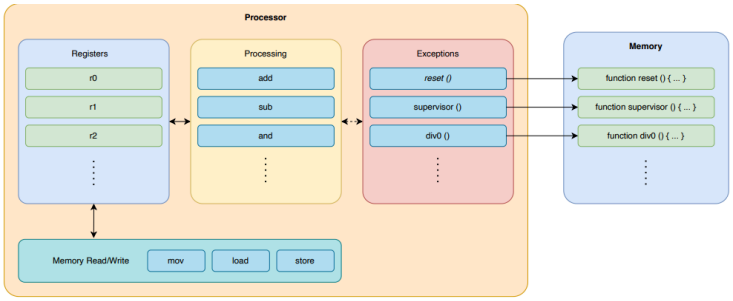
Pooling: checking the peripherals periodically(efficient for events in rapid succession, but keeps the processor busy)
Interrupt controller:
-each peripheral is connected to the interrupt controller
-The Interrupt controller receives interrupt requests, while the processor "does its thing"
-The Interrupt controller sets interrupt flags (that the processor can poll and decide when to handle)
Set up:
1. The processor initializes the peripheral and enables the interrupts it wants to receive from that peripheral
2. The processor initializes the interrupt controller with a list of priorities for each enabled interrupt.
3. There needs to be an interrupt service routine developed and placed in the program memory so the processor can execute it when needed.
4. Note: After the interrupt service routine finishes, the processor returns to normal code execution.

General flow:
1. Wait for the current instruction to complete
2. Save the Program Counter (PC) onto the stack
3. Save all active registers onto the stack
4. Jump to the address of the interrupt service routine (ISR) specified in the interrupt vector table
Note: depending on the architecture, the ISR might or might not be interruptable
5. Upon returning from the ISR, restore the saved registers and PC from the stack

On ATmega324P:
1. Complete the current instruction
2. Push (PC + 2) onto the stack
3. Push the SREG (Status Register) and any other general-purpose registers used in the main loop
4. Jump to the address in the interrupt vector table corresponding to the type of interrupt that occurred
5. In the table, there must be a jump to the Interrupt Service Routine (ISR)
6. Execute the ISR instruction

7. On return from ISR, pop the used general-purpose registers and then pop the SREG
8. Finally, pop (PC + 2) and resume execution in the main() loop

Exceptions:



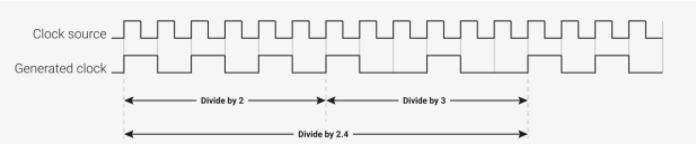
On embassy:
-Embassy registers handlers for Exceptions
-Each of the Embassy drivers that you use provides interrupt handlers for the peripheral they control
-Developers have to bind interrupts to the driver.

Timers

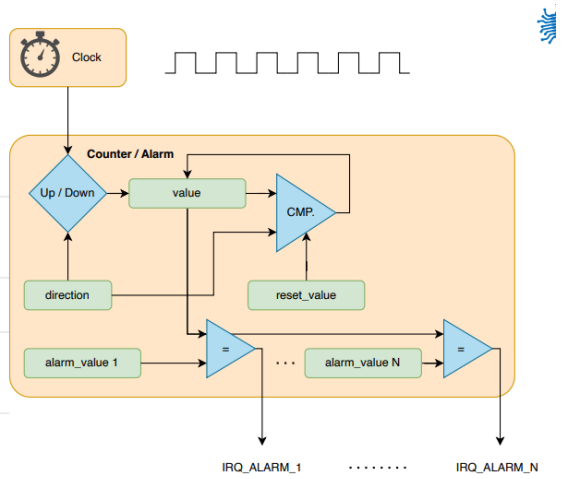
Problem(s):
- No single timer has the necessary resolution to time long intervals.
-If we set the prescaler to 1024 and use an 8-bit timer, then we can time at most 12MHz / 1024 / 256 = 45.77Hz (about 0.02s).

Watchdog:
-Sometimes the code loops and that is bad - as it will typically result in a freezer system.
-We will assume we have a code that might freeze and can accept a maximum of 4 seconds for the system to be stuck.
-the watchdog may, for example, save debug info on an eeprom and reset the controller if it has not been reset in 4 seconds

Frequency divider:
-divides down the clock signals used for the timer, giving reduced overflow rates
-allows the timer to be clocked at a user desired rate



Counter



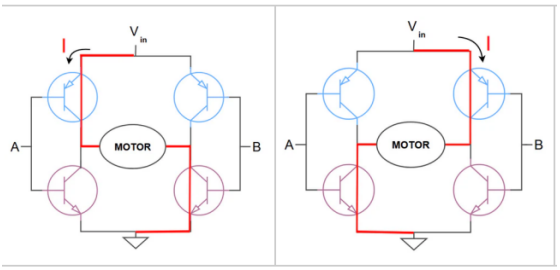
- **value**: the current value of the counter
- **direction**: set to count UP or DOWN
- **reset:** UP: the value at which the counter resets to 0 DOWN: the value to which the counter resets after getting to 0
- **alarm_x**: when value == alarm_x , triggers an interrupt, x in 1 … n

SysTick

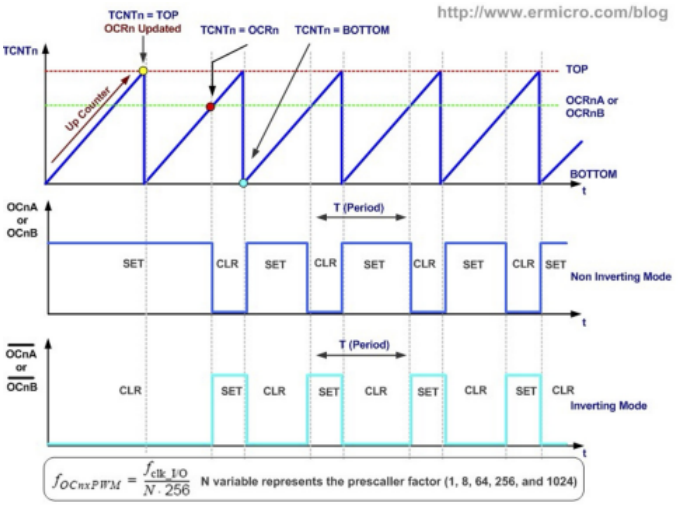
- decrements the value of SYST_CVR every µs
- when SYST_CVR becomes 0 :
  - triggers the SysTick exception
  - next clock cycle sets the value of SYST_CVR to SYST_RVR
- SYST_CALIB is the value of SYST_RVR for a 10ms interval (might not be available)

RP2350 Timers:
- stores 64 bits
- starts with 0
- increment every us
- has 4 alarms
- alarms registers are only 32 bits wide
- maximum 72 minutes

PWM

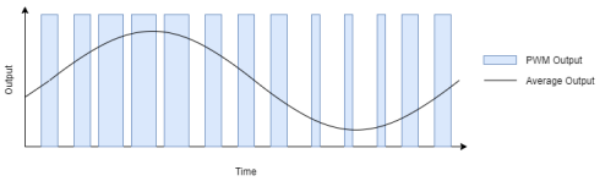


H Bridge for controlling motors with pwm



Duty Cycle Control via Code:
-The duty cycle is the percentage of time the signal is high (ON) in a full cycle
- It can be controlled from software by adjusting registers like OCRx (Output Compare Register);
-the higher the duty cycle, the higher the average voltage

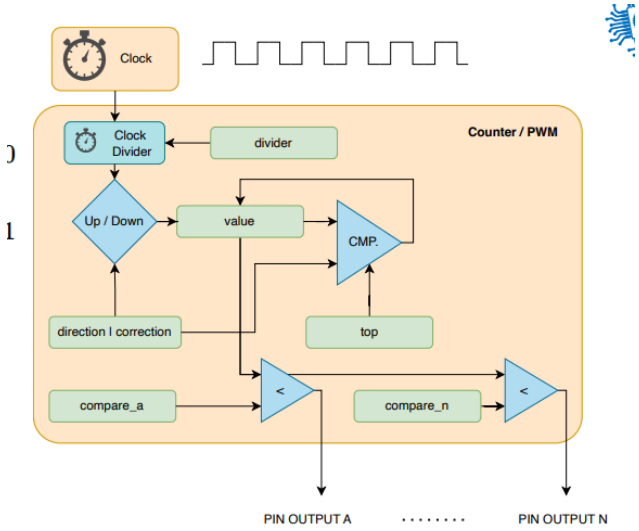
- generates a square signal
- if integrated (averaged), it looks like an analog signal



-frequency: :The number of repeats per s
- duty cycle: The percentage of the time when the signal is High

$$f = \frac{1}{period}\left[\frac{1}{s} = 1Hz\right]_{SI}$$

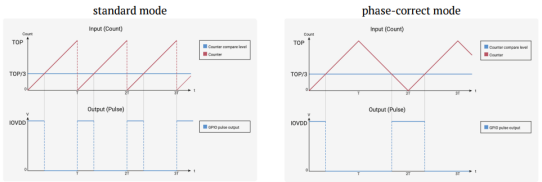
$$duty_cycle = \frac{time_on}{period}\%$$



$$f = \begin{cases} \frac{f_{clock}}{divider \times (top+1)} & correction = 0 \\ \frac{f_{clock}}{divider \times 2 \times (top+1)} & correction = 1 \end{cases}$$

$$pin_{a,b} = \begin{cases} 0 & compare_{a,b} >= value \\ 1 & compare_{a,b} < value \end{cases}$$

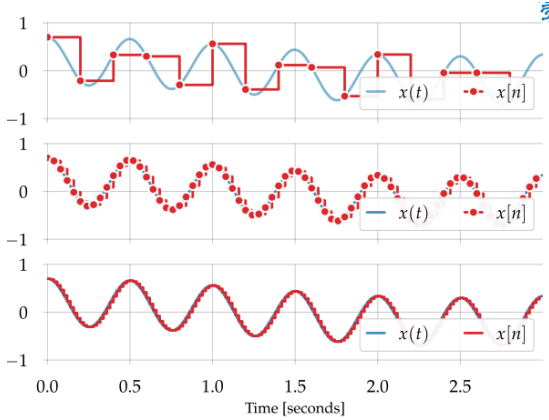
RP2350 PWM
-generates square signals
-counts the pulse width of input signals
-8 or 12 PWM slices, each A and B channels
-each PWM channel is linked to a fixed pin
-some channels are connected to two pins
-may be used as timers (IRQ1_INTE)



$$period = (TOP+1) \times (PH_CORRECT+1) \times \left(DIV_INT + \frac{DIV_FRAC}{16}\right)[s]_{SI}$$

ADC

- samplig rate: the frequency at which a new sample is read
- resolution: the number of bits used to store a sampled value



Nyquist–Shannon Sampling Theorem
-The sampling frequency has to be at least two times higher than the maximum frequency of the signal to avoid frequency aliasing .
- Aliasing is the overlapping of frequency components. This overlap results in distortion or artifacts when the signal is reconstructed from samples which causes the reconstructed signal to differ from the original continuous signal.(the first sampling)

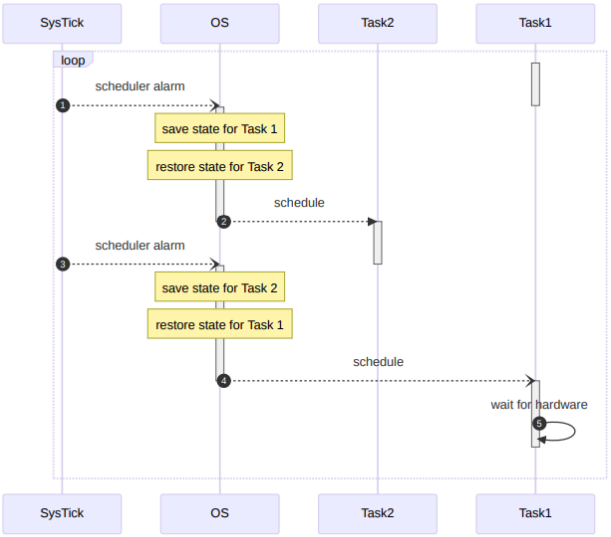
Sampling
-assumes bitn-1 of compare_value is 1
-compares the input signal with a generated analog signal from compare_value
-if input is lower, bit is 0 if inpu
-if higher, bit is 1
-repeats for every bit

RP2350 ADC
-channels: 4 or 8
-sampling rate: 500 kHz
- resolution: 12 bits
-Vmax: 3.3V
- requires a 48 MHz clock signal
- channel 4 or 8 is connected to the internal temperature sensor

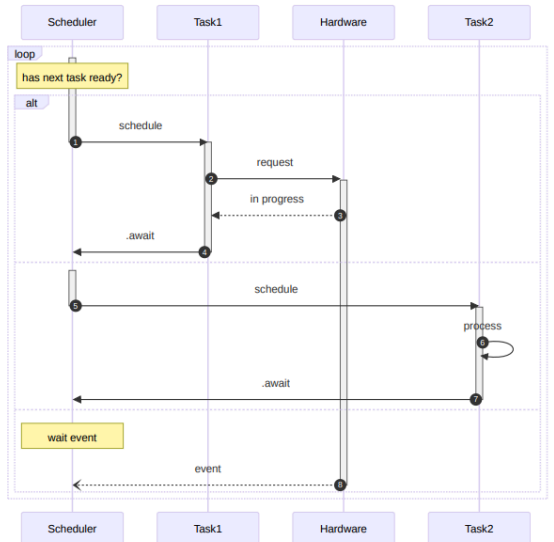
$$t = 27 - \frac{(V_{input_4} - 0.706)}{0.001721}[^{\circ}C]_{SI}$$

Concurrency

Preemptive Concurrency
-MCUs are usually single core
-Tasks in parallel require an OS
-Tasks can be suspended at any time
-Switching the task is expensive
-Tasks that do a lot of I/O which makes the switching time longer than the actual processing time

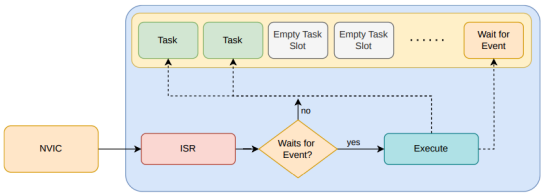


Cooperative Concurrency
-tasks cannot be interrupted
-hardware works in an asynchronous way
-tasks cooperate: give up the MCU for other tasks to use it while they wait for hardware
-there is no need for an OS, everything is done in one single flow
-no penalty for saving and restoring the state



Async in Embassy

-#[embassy_executor::main]
-starts the Embassy scheduler
defines the main task
-#[embassy_executor::task] - defines a new task
-pool_size -is optional and denes how many identical tasks can be spawned
-unless awaited, async functions are not executed
-tasks have to use .await in loops, otherwise they block the scheduler



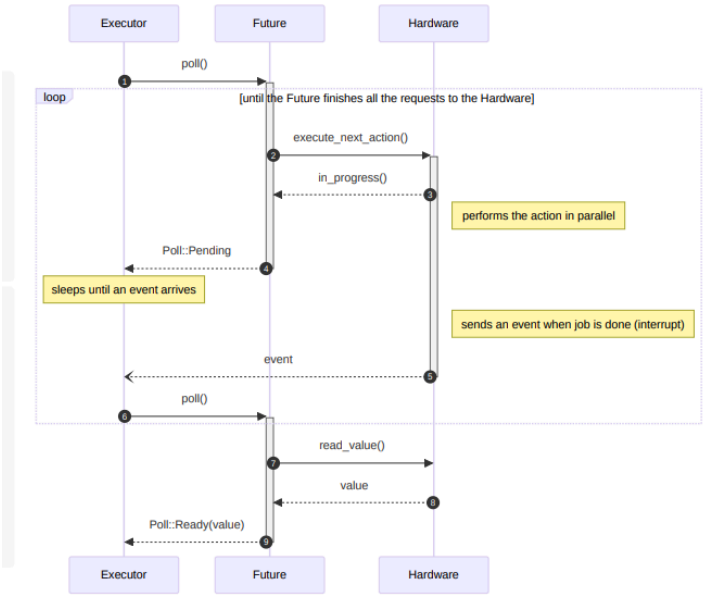
-sleep when all tasks wait for events
-after an ISR is executed
-if waiting for events, ask every task if it can execute (if the IRQ was what the task was .await ing for)
-if a task is executing, continue the task until it .await s
-if a task never .await s,the executor does not run and never executes another task

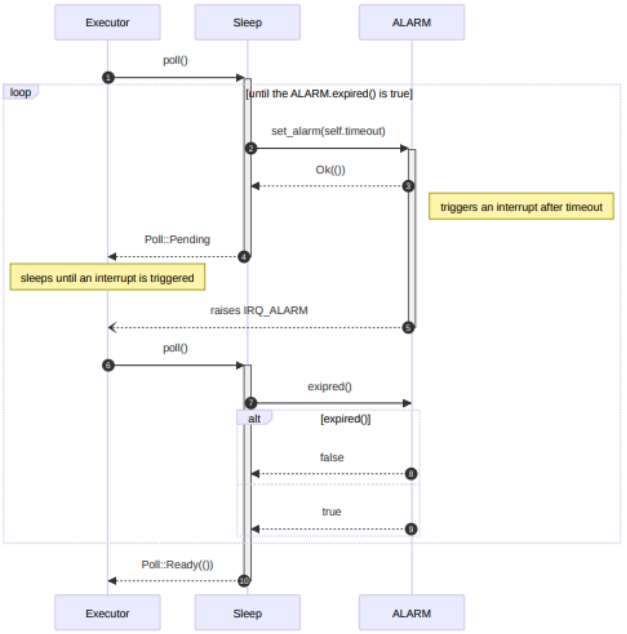
Priority Tasks

interrupt::SWI_IRQ_1.set_priority(Priority::P2);
interrupt::SWI_IRQ_0.set_priority(Priority::P3);

- priority executors run in ISRs, lower priority tasks are interrupted

Futures



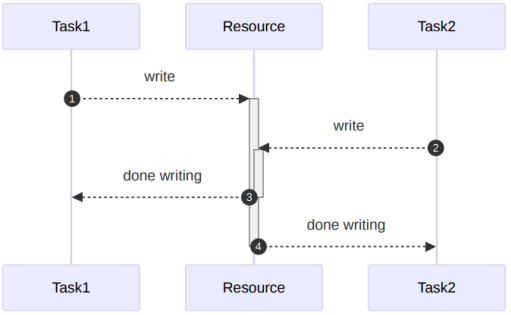


- the Rust compiler rewrites async function into Future
- it does not know how to execute them
- executors are implemented into third party libraries

Communication

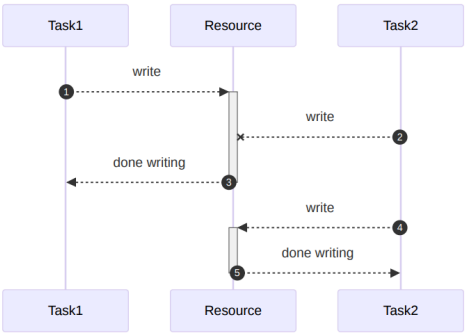
Simultaneous Access

Rust forbids simultaneous writes access



Exclusive Access

we want to sequentiality access the resource



Synchronization
- **NoopMutex** -used for data shared between tasks within the same executor
- **CriticalSectionMutex** -used for data shared between multiple executors,ISRs and cores
- **ThreadModeMutex** -used for data shared between tasks within low priority executors (not running in ISRs mode) running on a single core

Blocking Mutex: no .await allowed while the mutex is held

Async Mutex: .await is allowed while the Mutex is held, it will release the Mutex while await ing

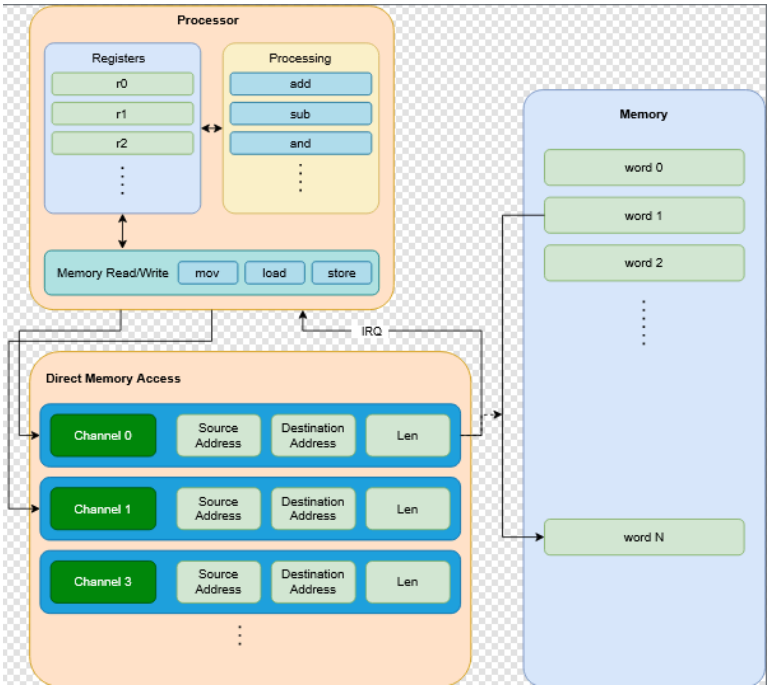
Type	Description
Channel	A Multiple Producer Multiple Consumer (MPMC) channel. Each message is only received by a single consumer
PriorityChannel	A Multiple Producer Multiple Consumer (MPMC) channel. Each message is only received by a single consumer. Higher priority items are shifted to the front of the channel.
Signal	Signalling latest value to a single consumer.
PubSubChannel	A broadcast channel (publish-subscribe) channel. Each message is received by all consumers.

DMA

- offloads the MCU from doing **memory to memory** operations
- due to MMIO, usually implies **transfers from and to peripherals**
- raises an interrupt when a transfer is done

⚠️ DMA does not know about the data stored in cache.

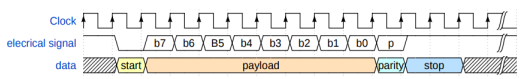
- for chips that use cache
  - the DMA buffer's memory region has to be set manually to nocache (if MCU knows)
  - or, the cache has to be flushed before and, possibly after, a DMA transfer



UART

aka serial port

- connects **two devices**
- uses two **independent** wires
  - *TX* - transmission wire
  - *RX* - reception wire
-
- cross-connected



bits	the number of bits in the payload, between 5 and 9
parity	add or not the parity bit
stop	the number of stop bits to add, 1 or 2
baud rate	number of elements sent per s, most used **9600** or **115200**

Baud rate

$$baud_{rate} = \frac{f_{clock}}{divider \times (1 + payload_{bits} + parity_{bits} + stop_{bits})}$$

UART Device

- **TTL** - *Transistor Transistor Logic* connects devices at 0 - 3.3V or 0 - 5V, used for short cables and jumper wires
- **RS232** - used for external connections and longer cables, uses -12V to 12V.
- **RS485** - industrial, uses differential voltage

Receiver - RX part of the serial port

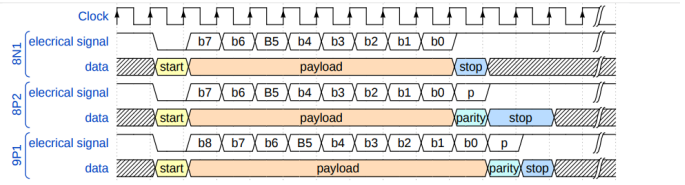
- *Shift Register* to read **serially every bit**
- Triggers an interrupt
  - when data was received
  - (*optional*) when FIFO is half full
  - (*optional*) when FIFO is full
- FIFO is optional
  - may have a capacity of 1

Transmiter - TX part of the serial port

- *Shift Register* to output **serially every bit**
- Triggers an interrupt
  - when data was sent
  - (*optional*) when FIFO is half empty
  - (*optional*) when FIFO is empty
- FIFO is optional
  - may have a capacity of 1

Example

Setup	Payload	Parity	Stop
8N1	8 bits	no	1 bit
8P2	8 bits	yes	2 bits
9P1	9 bits	yes	1 bit



Facts

Transmission	duplex	data can be sent in both directions at the same time
Clock	independent	there is no clock sent between the two devices, the **receiver** has to **synchronize its clock with the transmitter** to be able to correctly read the received data
Wires	TX/RX	one receive write, one transmit wire,

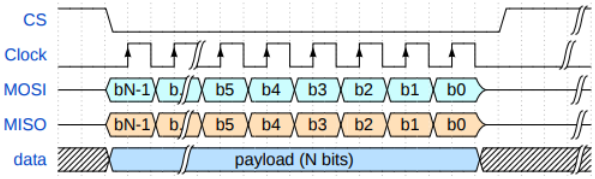
		independent of each other
Devices	2	a receiver and a transmitter
Speed	115 KB/s	usually a maximum baud rate of 115200 is used

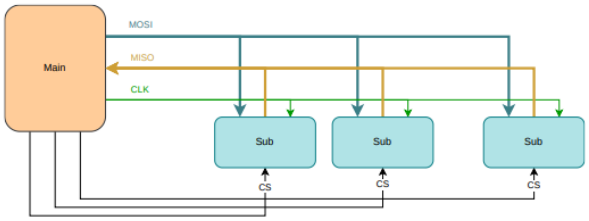
SPI(Serial Peripheral Interface)

- Used for communication between integrated circuits
- Sensors usually expose an SPI and an I2C interface
- Two device types:
  - main (master) - controls the communication (usually MCU)
  - sub (slave) - receive and transmit data when the main requests (usually the sensor)

Wires

- MOSI -Main Out Sub In- carries data from the main to the subs
- MISO -Main In Sub Out - carries data from the active sub to the main
- CLK-Clock - the clock signal generated by the main, subs sample and write data to the bus only on the clock edge
- CS*-Chip Select -not actually part of SPI, one wire / sub, activates one sub at a time
  - inactive subs have to disconnect from the MOSI and MISO lines





Example

1. main activates the sub device
   -sets the CS signal to LOW
2. at the same time
   -main puts the rst bit on the MOSI line
   -sub puts the rst bit on the MISO line
3. main starts the clock
4. at the rising edge
   - main reads the data from the MISO line
   -sub reads the data from the MOSI line
5. on the falling edge
   -main puts the next bit on the MOSI line
   -sub puts the next bit on the MISO line
6. repeat 4 and 5 until main decides to stop the clock

SPI Modes

Mode	CPOL	CPHA
0	0	0
1	0	1
2	1	0
3	1	1

`CPOL`	Clock polarity	defines when the data bit is read 0: *rising edge* 1: *falling edge*
`CPHA`	Clock phase	defines when the data is written to the line 0: when `CS` *activates* or *clock edge* 1: on *clock edge* (depends on `CPOL`)

Daisy Chaining

1. main activates all the subs
2. on the clock edge
   - main sends data to sub 1
   - sub 1 sends data to sub 2
   - …
   - sub n-1 sends data to sub n
   - sub n sends data to main

1. usually subs send the previous data bit received from main to the next sub

Facts

Transmission	duplex	data must be sent in both directions at the same time
Clock	synchronized	the main and sub use the same clock,there is no need for clock synchronization
Wires	MISO / MOSI / CLK / CS	different read and write wires, a clock wire and an optional chip select wire for every sub
Devices	1 main several subs	a receiver and a transmitter
Speed	no limit	does not have any limit, it is limited by the main clock and the electronics wirings

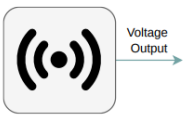
Usage

- EEPROMs / Flash (usually in QSPI mode)
  - Raspberry Pi Pico has its 2MB Flash connected using QSPI
- sensors
- small displays
- RP2350 has two SPI devices

Sensors

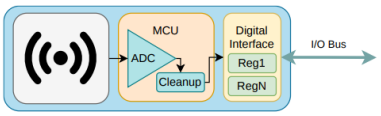
Analog

- only the transducer (the analog sensor)
- outputs (usually) voltage
- requires:
  - an ADC to be read
  - cleaning up the noise

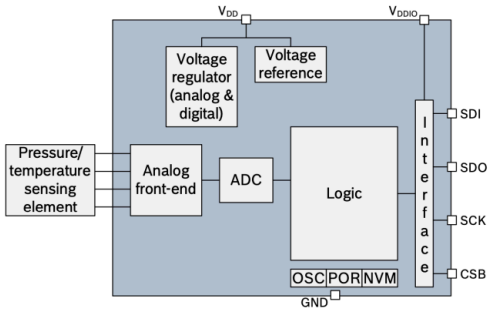


Digital

- consists of:
  - a transducer (the analog sensor)
  - an ADC
  - an MCU for cleaning up the noise
- outputs data using a digital bus

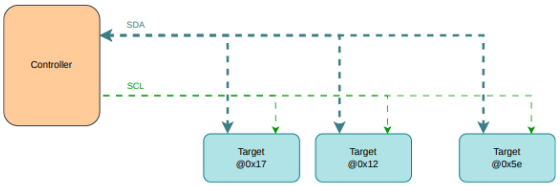


Example BME280



I2C(I square C)

- Used for communication between integrated circuits
- Sensors usually expose an SPI and an I2C interface
- Two device types:
    - controller (master) - initiates the communication (usually MCU)
    - target (slave) - receive and transmit data when the controller requests (usually the sensor)

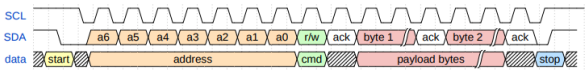


Wires and addresses

- SDA - Serial DAta line - carries data from the controller to the target or from the target to the controller
- SCL - Serial CLock line - the clock signal generated by the controller,targets
    - sample data when the clock is low
    - write data to the bus only when the clock is high
- each target has a unique address of 7 bits or 10 bits
- wires are never driven with LOW or HIGH
    - are always pull-up, which is HIGH
    - devices pull down the lines to write LOW

Example

1. controller issues a START condition
    - pulls the SDA line LOW
    - waits for ~ 1/2 clock periods and starts the clock
2. controller sends the address of the target
3. controller sends the command bit (R/W)
4. target sends ACK / NACK to controller
5. controller or target sends data (depends on R/W)
receives ACK / NACK after every byte
6. controller issues a STOP condition
    - stops the clock
    - pulls the SDA line HIGH while CLK is HIGH



Mode	Speed	Capacity	Drive Type	Direction
Standard mode (Sm)	100 kbit/s	400 pF	Open drain	Bidirectional
Fast mode (Fm)	400 kbit/s	400 pF	Open drain	Bidirectional
Fast mode plus (Fm+)	1 Mbit/s	550 pF	Open drain	Bidirectional
High-speed mode (Hs)	1.7 Mbit/s	400 pF	Open drain	Bidirectional
High-speed mode (Hs)	3.4 Mbit/s	100 pF	Open drain	Bidirectional
Ultra-fast mode (UFm)	5 Mbit/s	?	Push–pull	Unidirectional

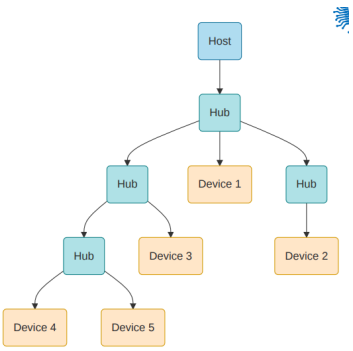
Facts

Transmission	half duplex	data must be sent in one direction at one time
Clock	synchronized	the controller and target use the same clock,there is no need for clock synchronization
Wires	SDA / SCL	dthe same read and write wire and a clock wire
Devices	1 controller several targets	a receiver and a transmitter
Speed	5 Mbit/s	usually 100 Kbit/s, 400 Kbit/s and 1 Mbit/s

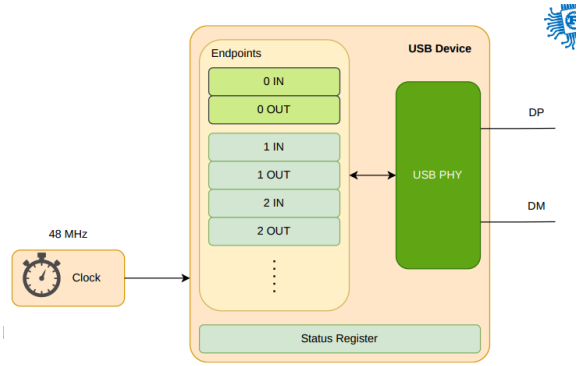
RP2350 has two I2C devices

USB 2.0(Universal Serial Bus)

- Used for communication between a host and several devices that each provide functions
- Two modes:
    - host - initiates the communication (usually a computer)
    - device - receives and transmits data when the host requests it
- each device has a 7 bit address assigned upon connect
    - maximum 127 devices connected to a USB host
- devices are interconnected using hubs
- USB devices tree



- can work as host or device, but not at the same time
- uses a differential line for transmission
- uses a 48 MHz clock
- maximum 16 endpoints (buffers)
    - IN - from device to host
    - OUT - from host to device
- endpoints 0 IN and OUT are used for control

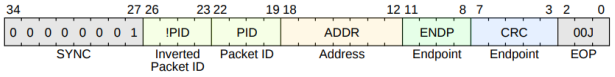


Token Packet

- usually asks for a data transmission

Type	PID	Description
OUT	0001	host wants to transmit data to the device
IN	1001	host wants to receive data from the device
SETUP	1101	host wants to setup the device

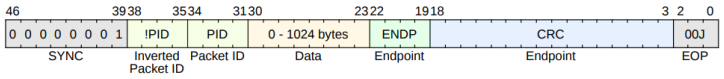
Address: `ADDR : ENDP`



Data Packet

- transmits data(between 0 and 1024 bytes)

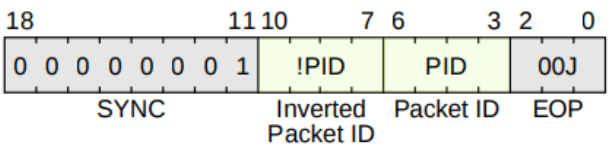
Type	PID	Description
DATA0	0011	the data packet is the first one or follows after a DATA1 packet
DATA1	1011	the data packet follows after a DATA0 packet



Handshake Packet

- acknowledges data

Type	PID	Description
ACK	0010	data has been successfully received
NACK	1010	data has not been successfully received
STALL	1110	the device has an error



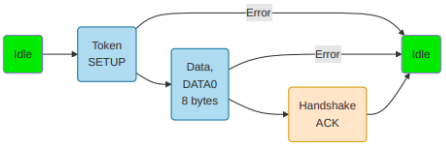
Transmission Modes

- Control -used for conguration
- Isochronous -used for high bandwidth, best effort
- Bulk -used for low bandwidth, stream
- Interrupt -used for low bandwidth, guaranteed latency

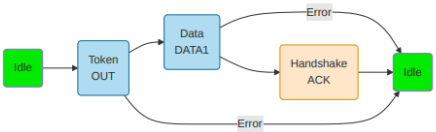
Control Mode

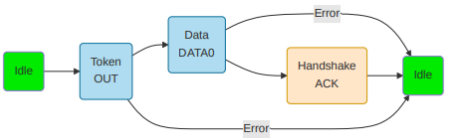
- used to control a device - ask for data

Setup - send a command (*GET_DESCRIPTOR*, ...)



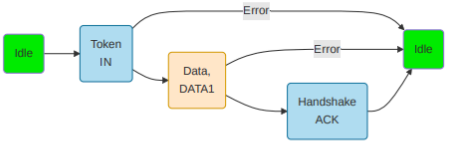
Data - *optional* several transfers, host transfers data





...

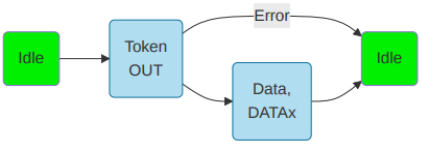
Status - report the status to the host



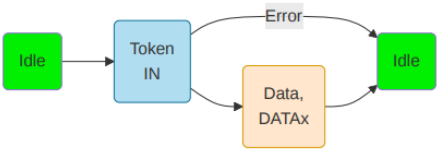
Isochronous Mode

- fast but not reliable transfer
- has a guaranteed bandwidth
- allows data loss
- used for functions like streaming where loosing a packet has a minimal impact

OUT - transfer data from the host to the device



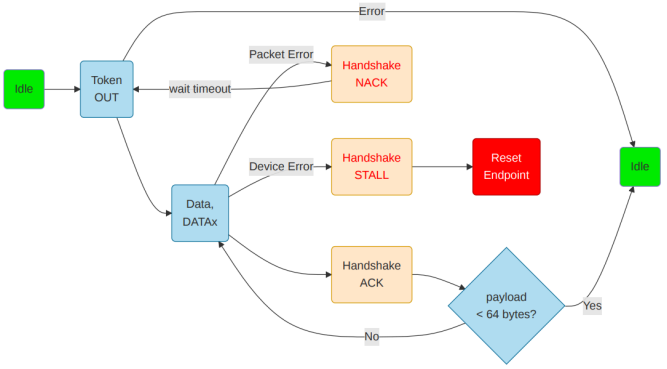
IN - transfer data from the device to the host



Bulk Mode

- slow, but reliable transfer
- does not have a guaranteed bandwidth
- does not allow data loss
- used for large data transfers where loosing packets is not permitted

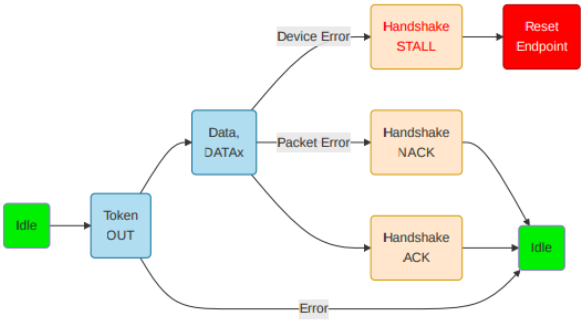
OUT - transfer data from the host to the device
IN- transfer data from the device to the host



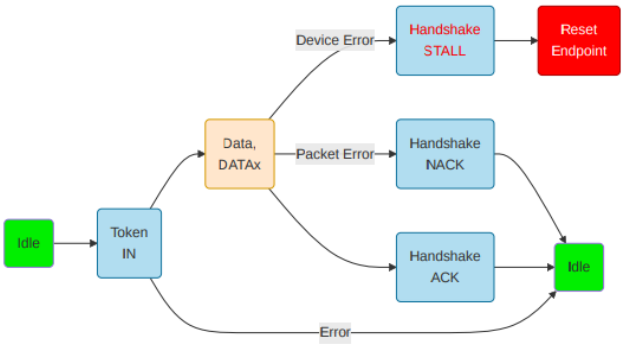
Interrupt Mode

- transfer data at a minimum time interval
- the endpoint descriptor asks the host start an interrupt transfer at a time interval
- used for sending and receiving data at certain intervals

OUT - transfer data from the host to the device



IN - transfer data from the device to the host



Device Organization

- a device can have multiple congurations
  - for instance different functionality based on power consumption
- a conguration has multiple interfaces
  - a device can perform multiple functions
  - Debugger
  - Serial Port
- each interface has alternate settings with multiple endpoints attached
  - endpoints are used for data transfer
  - maximum 16 endpoints, can be congured IN and OUT
- the device reports the descriptors in this order

Device Classes

- predefined device types

Device Class(0x00), Audio(0x01), Communications and CDC Control(0x02), HID (Human Interface Device)(0x03), Physical Interface Device (PID)(0x05), Image(0x06), Printer(0x07), Mass Storage(0x08), Still Image Capture Device(0x0A), Smart Card(0x0B), Content Security(0x0D), Video(0x0E), Personal Healthcare(0x0F), Audio/Video(0x10), Health Device(0x11), Diagnostic Device(0x12), Vendor Specific(0xFF)

Device Descriptor

- describes the whole device

Field	Value	Description
bLength	18	Descriptor length in bytes.
bDescriptorType	1	Descriptor type (1 = Device Descriptor).
bcdUSB	0x0200	USB specification release number (2.0).
bDeviceClass	0xFF	Device class (0xFF = Vendor Specific).
bDeviceSubClass	0	Device subclass (0 = defined by the interface).
bDeviceProtocol	0	Device protocol (0 = defined by the interface).
bMaxPacketSize0	64	Maximum packet size for endpoint 0 (64 bytes).
idVendor	0xCODE	Vendor ID (example: 0xCODE).
idProduct	0xCAFE	Product ID (example: 0xCAFE).
bcdDevice	0x0100	Device release number (example: 1.0).
iManufacturer	1	Index of the string descriptor for the manufacturer.
iProduct	2	Index of the string descriptor for the product.
iSerialNumber	3	Index of the string descriptor for the serial number.
bNumConfigurations	1	Number of configurations supported by the device.

Configuration Descriptor

- one of the configurations

Field	Value	Description
bLength	9	Descriptor length in bytes (always 9 for configuration descriptor).
bDescriptorType	2	Descriptor type (2 = Configuration Descriptor).
wTotalLength	0x0022	Total length of data returned for this configuration (including all descriptors).
bNumInterfaces	1	Number of interfaces supported by this configuration.
bConfigurationValue	1	Value to select this configuration.
iConfiguration	4	Index of the string descriptor describing the configuration.
bmAttributes	0x80	Configuration characteristics (bus-powered, no remote wake-up).
bMaxPower	50	Maximum power consumption (in 2mA units, so 50 means 100mA).

Interface Descriptor

Field	Value	Description
bLength	9	Descriptor length in bytes (always 9 for interface descriptor).
bDescriptorType	4	Descriptor type (4 = Interface Descriptor).
bInterfaceNumber	0	Number of this interface (starting from 0).
bAlternateSetting	0	Alternate setting (0 = default setting).
bNumEndpoints	1	Number of endpoints used by this interface.

bInterfaceClass	0xFF	Interface class (0xFF = Vendor Specific).
bInterfaceSubClass	0	Interface subclass (0 = vendor specific).
bInterfaceProtocol	0	Interface protocol (0 = vendor specific).
iInterface	5	Index of the string descriptor describing this interface.

Endpoint descriptor

Field	Value	Description
bLength	7	Descriptor length in bytes (always 7 for endpoint descriptor).
bDescriptorType	5	Descriptor type (5 = Endpoint Descriptor).
bEndpointAddress	0x81	Endpoint address (0x81): Bit 7 = IN direction (device to host), Bits 0–3 = endpoint number (1).
bmAttributes	0x02	Endpoint attributes (0x02 = Bulk endpoint).
wMaxPacketSize	64	Maximum packet size the endpoint can handle (64 bytes).
bInterval	0	Interval for polling (relevant for interrupt endpoints; 0 for others).

String Descriptor

- This string descriptor corresponds to Conguration 1. The string is encoded in UTF-16LE (littleendian). Each character is represented by two bytes.

Field	Value	Description
bLength	7	Descriptor length in bytes (always 7 for endpoint descriptor).
bDescriptorType	5	Descriptor type (5 = Endpoint Descriptor).
bEndpointAddress	0x81	Endpoint address (0x81): Bit 7 = IN direction (device to host), Bits 0–3 = endpoint number (1).
bmAttributes	0x02	Endpoint attributes (0x02 = Bulk endpoint).
wMaxPacketSize	64	Maximum packet size the endpoint can handle (64 bytes).
bInterval	0	Interval for polling (relevant for interrupt endpoints; 0 for others).

USB Modes

Mode	Speed	Version
Low Speed	1.5 Mbit/s	1.0
Full Speed	12 Mbit/s	1.0
High Speed	480 Mbit/s	2.0

Facts

Transmission	half duplex	data must be sent in one direction at one time
Clock	synchronized	the host and the device must synchronize their clocks
Wires	DP / DM	data is sent in a differential way
Devices	1 host several devices	a receiver and a transmitter
Speed	480 MBbit/s	

Certificari

IPXY (International Protection sau Ingress Protection)

● Prima cifra: rezistenta la praf, a doua cifra: rezistenta la apa

Level sized	Effective against	Description
0	—	No protection against contact and ingress of objects
1	>50 mm	Any large surface of the body, such as the back of a hand, but no protection against deliberate contact with a body part
2	>12.5 mm	Fingers or similar objects
3	>2.5 mm	Tools, thick wires, etc.
4	>1 mm	Most wires, slender screws, large ants etc.
5	Dust protected	Ingress of dust is not entirely prevented, but it must not enter in sufficient quantity to interfere with the satisfactory operation of the equipment.
6	Dust tight	No ingress of dust; complete protection against contact (dust tight). A vacuum must be applied. Test duration of up to 8 hours based on air flow.

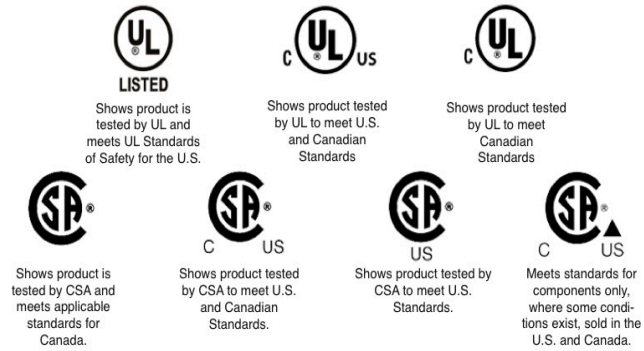
Level	Protection against
0	None
1	Dripping water
2	Dripping water when tilted at 15°
3	Spraying water
4	Splashing of water
5	Water jets
6	Powerful water jets
6K	Powerful water jets with increased pressure
7	Immersion, up to 1 m depth
8	Immersion, 1 m or more depth
9K	Powerful high temperature water jets

CE - uz general

UE + Tarile Nordice
● Businesses know that products bearing the CE marking can be traded in the EEA without restrictions.
● Consumers enjoy the same level of health, safety, and environmental protection throughout the entire EEA.
● Produsele trebuie sa respecte reglementarile directivelor

FCC - uz general USA

FCC se aplica pentru dispozitive electronice cu oscilatii de peste 9KHz
● FCC este echivalent cu directiva europeana CISPR 22
● Doua clase de certificare: Clasa A pentru echipamente industriale (mai permisiva), Clasa B pentru home-use (mai restrictiva)
● Pentru dispozitivele ce se folosesc la priza – standardele UL

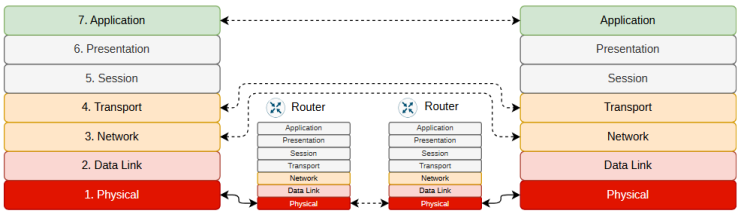


Mediu

ROHS - Restriction of Hazardous Substances Directive
WEEE - Waste Electrical and Electronic Equipment

Wi-Fi

OSI



L1 hardware, sends and receives data on the physical media
L2 hardware and driver sends and receives data from a device that it is directly connected to
L3 driver sends and receives data from devices not directly connected to using L2 from device to device
L4 driver connects the applications to the networking stack
L5/L6 not used
L7 is the application

Wi-Fi

L2 (Data Link) Protocol
Devices
AP - Access Point - acts as a hub or switch handles authentication
Device - The device that connects to the network
Frequencies : 2.4 GHz & 5 GHz

Wireless Network Connection

Open - everyone receives all the communication
WEP - all data is encrypted with the same key, everyone who knows the keys can read the data
WPA 1/2/3 (Personal) - each device has a different encryption key shared with the AP

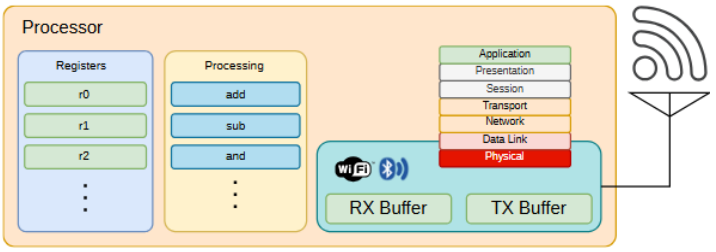
- the device authenticates with the AP by using the network passkey
- the device and the AP exchange a symmetric encryption key

WPA 1/2/3 Enterprise - each device has a different encryption key shared with the AP

- the AP provides a certificate to the device proving its authenticity
- the device authenticates using username and password or a private key
- the device and the AP exchange a symmetric encryption key

Integrated Network Device

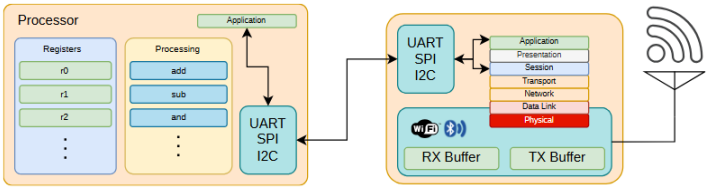
- a radio peripheral
- knows how to emit and receive in 2.4 and 5 GHz
- is controlled by software
- can generate signals for Wi-Fi, BLE, 802.15.4, 6LoPAN, Thread
- it knows how to transmit and receive buffers (L1), some devices know L2



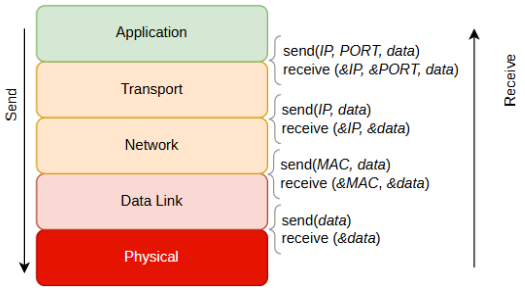
Discrete Network Device

the MCU is connected to an external Wi-Fi/BLE device transport over UART, SPI or I2C. Most devices knows

- L3 - provides socket
- L4 - provides TCP/UDP sockets
- L7 - provides application functions (usually HTTP and MQTT)



TCP/IP Stack



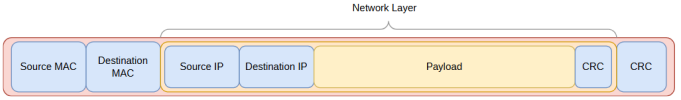
Data Link Layer

- very similar for Ethernet and Wi-Fi (HDLC)
- uses *Media Access Control* (*MAC*) addresses
- sends and receives *frames* from other devices directly connected to the same network



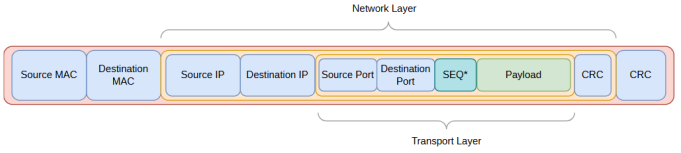
Network Layer

- Internet Protocol
- uses *Internet Protocol* (IP) addresses
  - *IPv4* - 32 bits
  - *IPv6* - 128 bits
- sends and receives *packets* from other devices remotely



Transport Layer

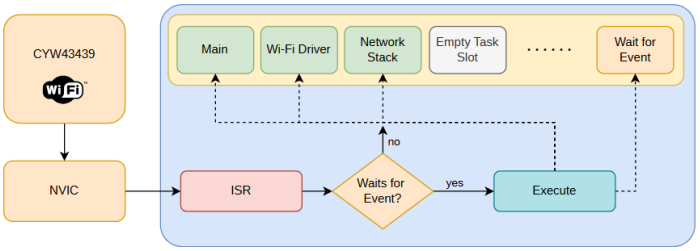
- Two protocols
  - *Transport Control Protocol* (*TCP*) - stream of data, makes sure it gets to the destination
  - *User Datagram Protocol* (*UDP*) - *fire and forget*, best effort do deliver the packet
- uses *Ports* to identify the destination and source application
- sends and receives *packets*



Pico W

- Wi-Fi and BLE provided by CYW43439 made by Infineon
- connected over SPI/PI0
- Wi-Fi 4 (802.11n), 2.4 GHz
- WPA 3
- SoftAP (4 clients)
- BLE 5.2
- Bluetooth Classic
- Provides L2 - allows sending of Ethernet (MAC)

frames



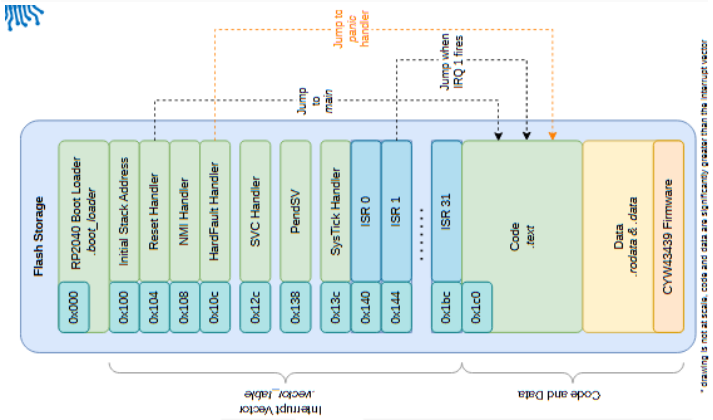
CYW43439

Load the firmware into the .data section.

```
let fw =
include_bytes!("./cyw43439_firmware/43439A0.bin");
let clm =
include_bytes!("./cyw43439_firmware/43439A0_clm.bin");
```

Pio0 as SPI

```
bind_interrupts!(struct Irqs {
PIO0_IRQ_0 => InterruptHandler<PIO0>;
});
let pwr = Output::new(p.PIN_23, Level::Low);
let cs = Output::new(p.PIN_25, Level::High);
let mut pio = Pio::new(p.PIO0, Irqs);
let spi = PioSpi::new(
&mut pio.common, pio.sm0, pio.irq0,
cs, p.PIN_24, p.PIN_29, p.DMA_CH0
);
```



Task

```
#[embassy_executor::task]
async fn wifi_task(runner: cyw43::Runner<'static,
Output<'static>, PioSpi<'static, PIO0, 0, DMA_CH0>>) -> !
{
runner.run().await
}
```

Start Device

```
static STATE: StaticCell<cyw43::State> =
StaticCell::new();
 let state = STATE.init(cyw43::State::new());
 let (_net_device, mut control, runner) =
cyw43::new(state, pwr, spi, fw).await;
 unwrap!(spawner.spawn(wifi_task(runner)));
control.init(clm).await;
control.set_power_management(PowerManagementMode::PowerSa
```

```
ve).await;
```

The first action of the wifi task is to write the firmware from .data to the CY43439 chip.

```
#[embassy_executor::task]
async fn wifi_task(/* ... */) -> ! {
runner.run().await
}
```

AP Mode

Open: network SSID, channel number

```
control.start_ap_open("Network SSID", 5).await;
```

WPA network: network SSID, WPA password, channel number

```
control.start_ap_wpa2("Network SSID", "WPA password",
5).await;
```

Device Mode

Open Network: network SSID

```
control.join_open("network SSID").await;
```

WPA network: network SSID, network password

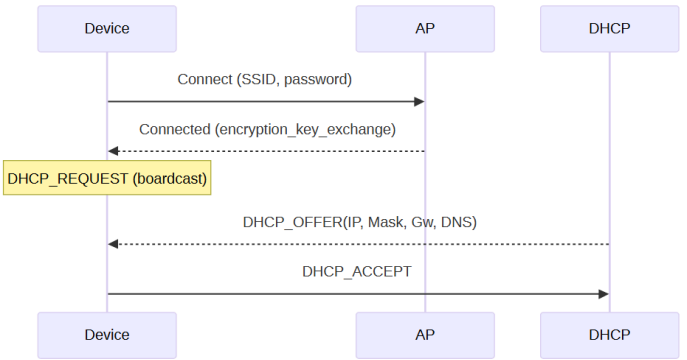
```
control.join_wpa2("network ssid", "network
password").await {
 Ok(_) => break,
 Err(err) => {
    info!("join failed with status={}", err.status);
  }
}
```

Embassy Net

- uses smoltcp, embedded (no_std) TCP/IP stack, written in Rust
- L3: IPv4, IPv6, IGMPv4 (ping), 6LoWPAN
- L4: TCP and UDP
- L7: DHCPv4 and DNS

DHCP

Dynamic Host Control Protocol



Network code

Self assigned

```
let config =
embassy_net::Config::ipv4_static(embassy_net::StaticConfi
gV4 {
address: Ipv4Cidr::new(Ipv4Address::new(192, 168, 69, 2),
24),
dns_servers: vec![Ipv4Address::new(8, 8, 8, 8),
Ipv4Address::new(1, 1, 1, 1)],
gateway: Some(Ipv4Address::new(192, 168, 69, 1)),
 });
```

Dynamic Host Control Protocol (DHCP)

```
let config = Config::dhcpv4(Default::default());23 //
start the network stack
 // Wait for DHCP
```

```
info!("waiting for DHCP...");
while !stack.is_config_up() {
Timer::after_millis(100).await;
}
```

Start Network Stack

```
// chosen by fair dice roll. guaranteed to be random.
let seed = 0x0123_4567_89ab_cdef;
// Init network stack static STACK
StaticCell<Stack<cyw43::NetDriver<'static>>> =
StaticCell::new();
static RESOURCES: StaticCell<StackResources<2>> =
StaticCell::new();
let stack = &*STACK.init(Stack::new( net_device, config,
RESOURCES.init(StackResources::<2>::new()), seed, ));
unwrap!(spawner.spawn(net_task(stack)));
```

DNS

```
let dns = DnsSocket::new(stack);
match dns.get_host_by_name("www.example.com",
AddrType::IPv4) {
Ok(ip) => info!("Ip is {:?}", ip),
Err(e) => warn!("failed to retrieve address {:?}", e)
}
```

TCP SERVER

```
let mut rx_buffer = [0; 4096];
let mut tx_buffer = [0; 4096];

loop {
let mut socket = TcpSocket::new(stack, &mut rx_buffer,
&mut tx_buffer);
socket.set_timeout(Some(Duration::from_secs(10)));
info!("Listening on TCP:1234...");
if let Err(e) = socket.accept(1234).await {
    warn!("accept error: {:?}", e);
    continue;
}
info!("Received connection from {:?}",
socket.remote_endpoint());
```

TCP CLIENT

```
let mut rx_buffer = [0; 4096];
let mut tx_buffer = [0; 4096];

loop {
let mut socket = TcpSocket::new(stack, &mut rx_buffer,
&mut tx_buffer);
socket.set_timeout(Some(Duration::from_secs(10)));
info!("Connecting to TCP 1.2.3.5:1234...");
if let Err(e) =
socket.connect(IpEndpoint::new(IpAddress::v4(1,2,3,5),
1234)).await {
    warn!("accept error: {:?}", e);
    continue;
}
info!("Connected to {:?}", socket.remote_endpoint());
```

READ FROM TCP SOCKET

```
let mut buf = [0u8; 4096];

let n = match socket.read(&mut buf).await {

    Ok(0) => {
        warn!("read EOF");
        break;
    }
    Ok(n) => n,
    Err(e) => {
        warn!("read error: {:?}", e);
        break;
    }
};
info!("rxd {}", from_utf8(&buf[..n]).unwrap());
```

WRITE TO A TCP SOCKET

```
let buf = [0u8; 4096];

match socket.write_all(&buf[..n]).await {
    Ok(()) => {
        /* write was a success */
    }
    Err(e) => {
         warn!("write error: {:?}", e);
         break;
     }
};
```

LISTEN FOR UDP PACKETS

```
let mut rx_buffer = [0; 4096];
let mut rx_metdata_buffer = [PacketMetadata::EMPTY; 3];
let mut tx_buffer = [0; 4096];
let mut tx_metadata_buffer = [PacketMetadata::EMPTY; 3];

let mut buf = [0u8; 4096];

let mut socket = UdpSocket::new(
stack, &mut rx_buffer, &mut rx_metadata_buffer, &mut
tx_buffer, &mut tx_metadata_buffer);

// listen for UDP packet on port 1234
if let Err(e) = socket.bind(1234) {
    warn!("bind error {:?}", e);
    break;
}
```

SEND UDP PACKETS

```
let mut rx_buffer = [0; 4096];
let mut rx_metdata_buffer = [PacketMetadata::EMPTY; 3];
let mut tx_buffer = [0; 4096];
let mut tx_metadata_buffer = [PacketMetadata::EMPTY; 3];

let mut buf = [0u8; 4096];

let mut socket = UdpSocket::new(stack, &mut rx_buffer,
&mut rx_metadata_buffer, &mut tx_buffer, &mut
tx_metadata_buffer);
```

Libraries
- **MQTT** - MQ Telemetry Transport
  - publish/subscribe
  - minimq
- **CoAP** - Constrained Application Protocol
  - simplified binary HTTP
  - coap_lite

ZigBee

- Technological standard created for the control and sensor networks
- Suited for low-cost, low-power wireless IoT networks
- Based on IEEE 802.15.4 standard
- Created by ZigBee Alliance – Philips, Motorola, HP, Intel etc.
- Conceived in 1998, standardized in 2003, and revised in 2006

Perfromance

- Proven excellent in-building coverage
  - Inherently robust radio link
  - Mesh networking
  - Acknowledge oriented protocol
- Proven tolerance to interference

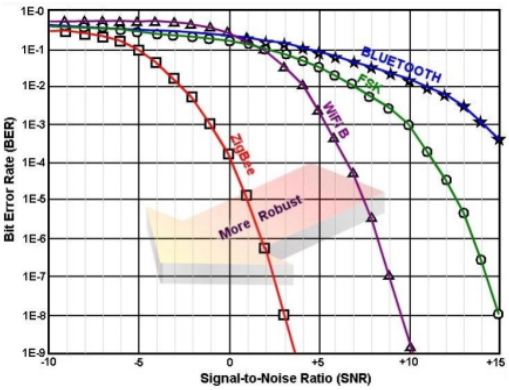
- Trade shows like CES-works when WiFi and Bluetooth fail
- Montage Hotels and MGM City Center deployments
- Products which implement multiple radio technologies

- Proven coexistence
  - Many multi-radio products and multi-radio deployments
- Proven scalability
  - City Center at 70,000 plus radios
  - Montage Hotels at 4000 plus radios per property

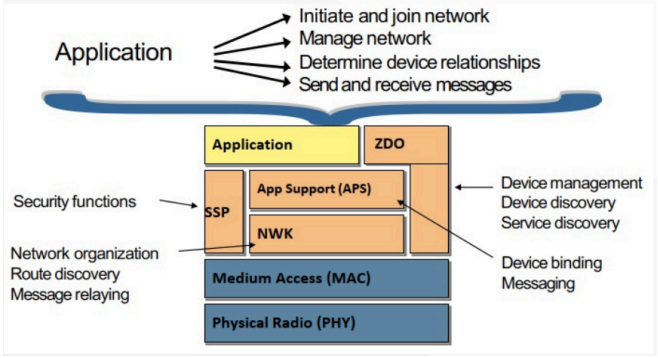
Characteristics

- 65,536 network (client) nodes
- 27 channels over 2 bands
- 250Kbps data rate
- Optimized for timing-critical applications and power management
- Full Mesh Networking Support
- ZigBee technology relies upon IEEE 802.15.4, which has excellent performance in low SNR environments

Frequency Band	License Required?	Geographic Region	Data Rate	Channel Number(s)
868.3 MHz	No	Europe	20kbps	0
902-928 MHz	No	Americas	40kbps	1-10
2405-2480 MHz	No	Worldwide	250kbps	11-26



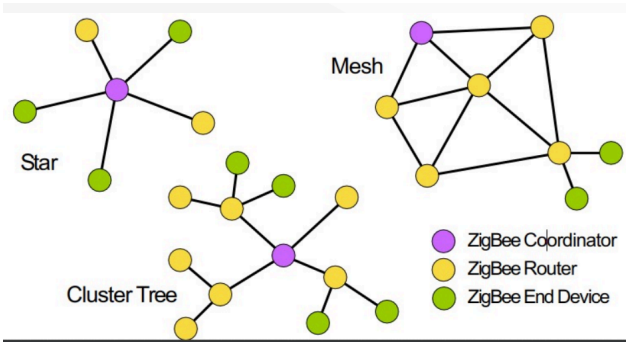
Zigbee Stack



Device Types

- ZigBee Coordinator (ZC)
  - One required for each ZB network.
  - Initiates network formation.
- ZigBee Router (ZR)
  - Participates in multihop routing of messages.
- ZigBee End Device (ZED)
  - Does not allow association or routing.
  - Enables very low cost solutions

Network Topologies

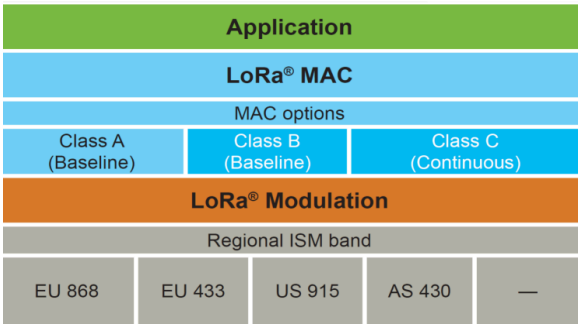


Public Profiles

- Home Automation (HA)
- Smart Energy (SE)
- Commercial Building Automation (CBA)
- ZigBee Health Care (ZHC)
- Telecom Applications (TA)
- ZigBee RF4CE Remote Control
- +Future profiles proposed by member companies…

LoRaWan

- LoRaWAN is a wireless technology designed for low-power WAN networks with low cost, mobility, security, and bidirectional communication for IoT applications
- It is a low-power consumption optimized protocol designed for scalable wireless networks with millions of devices
- It supports redundant operation, location free, low cost, low power and energy harvesting technologies to support the future needs of IoT while enabling mobility and ease of use features

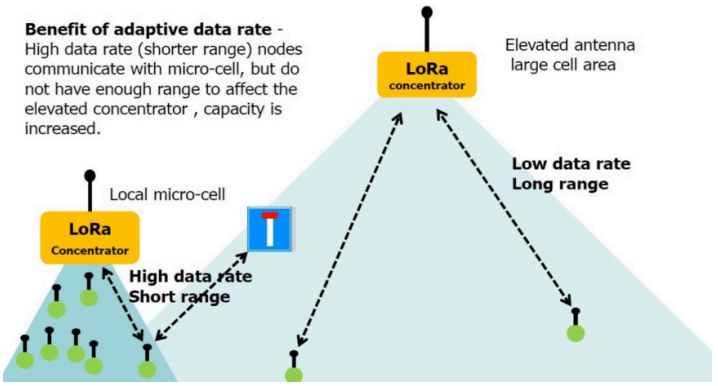


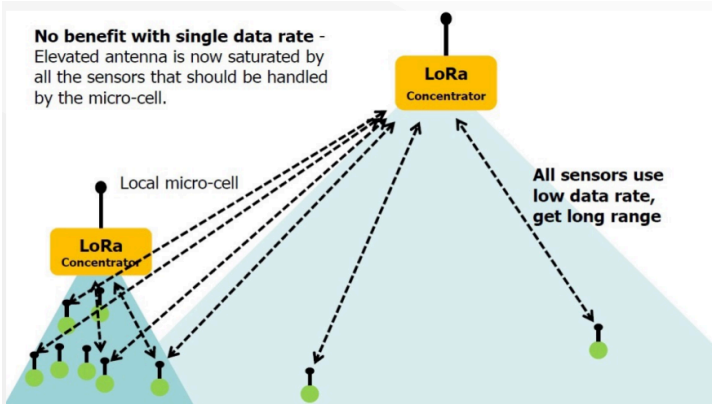
Two major components

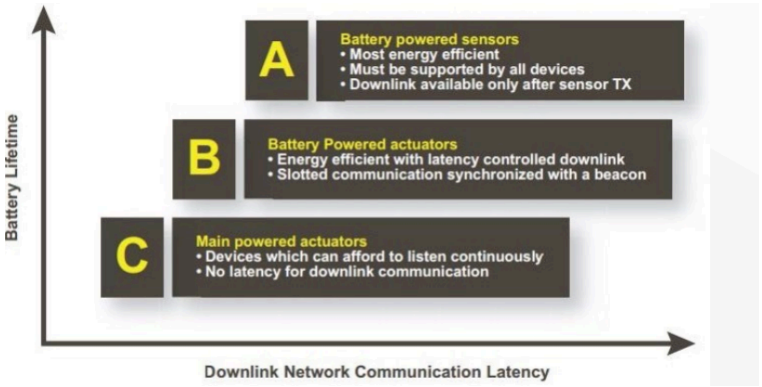
- End device: ED
- Base Station: BS

Network Protocol:

- LoRaWAN network protocol is optimized specifically for energy limited EDs
- LPWAN typically has star topology and consists of BSs relaying data messages between the EDs and an application server
- The BSs can be connected to the central server via backbone internet protocol (IP) based link, and the wireless communication based on LoRa or GFSK modulation is used to move the data between EDs and the BSs







SIGFOX

- Deploy its own base stations to support IoT applications
  - Kilo meter communication distance
  - Connect thousands of devices
  - 100 bits per second date rate
  - 5 years life time
- REPETITION OF THE MESSAGE
  - Each message sent 3 times
  - Repetition at 3 different time slot = time diversity
  - Repetition at 3 different frequencies = frequency diversity
- COLLABORATIVE NETWORK
  - Network deployed and operated to have 3 base stations coverage at all times = space diversity
- MINIMIZATION OF COLLISIONS
  - Probability of collisions are highly reduced
  - Ultra Narrow Band
  - 3 base stations at 3 different locations

NB-IoT(Ultra Narrow Band)

- Reduce the transmitted signal bandwidth
  - Reduced noise power
  - Therefore, we can reduce the transmission power
  - Therefore, we can reduce the power consumption of radio communication

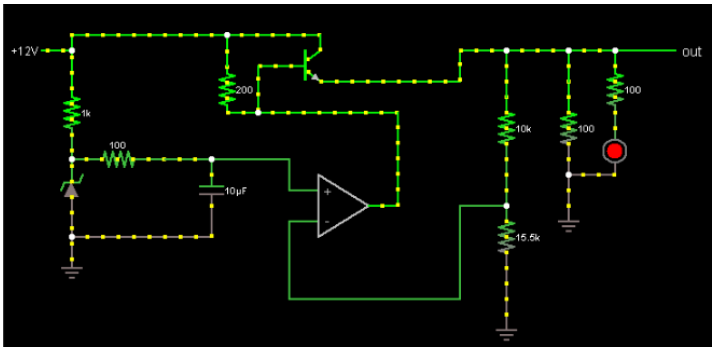
Power

Stabilizator de tensiune

Protejează împotriva fluctuațiilor de tensiune
Le putem utiliza la dispozitivele din locuință: Centrale termice, Aer condiționat, Televizoare, Calculatoare
Sau in cadrul proiectelor de la PM (DC-to-DC)
• Previn defecțiuni nedorite sau funcționări defectuoase

Stabilizator de tensiune liniar

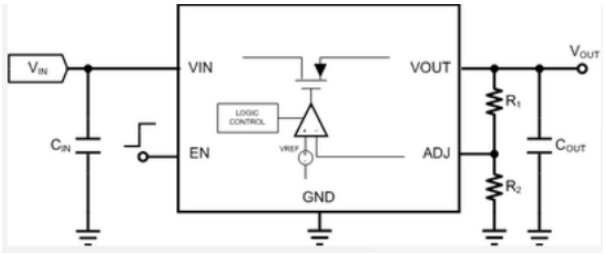
• Schemă interna simplificată
• Utilizează o dioda (Zener) sau tranzistor ca element activ
• Funcție de rezistor variabil ce permite reglarea tensiunii de ieșire
• Excesul de energie e disipat ca și căldură



Parametrii relevanți
• Voltage dropout : V_DROP = V_IN(min) – V_OUT
• Quiescent current : Curentul consumat intern de stabilizator pentru a-și menține funcționarea (curentul „de **auto-consum**").Se adaugă la curentul consumat de sarcină.
• Current limit : Valoarea maximă de curent pe care o poate furniza stabilizatorul înainte să intre în protecție (foldback sau shutdown).
• PSSR (Power-supply rejection ratio) : PSSR(f) = 20 * log10 (ΔV_IN(f) / ΔV_OUT(f)) - cu cât **mai mare**, cu atât **mai bine**.
• Output noise : Zgomotul electric generat de stabilizator însuși la ieșire (independent de zgomotul de la intrare).

LDO - Low Dropout

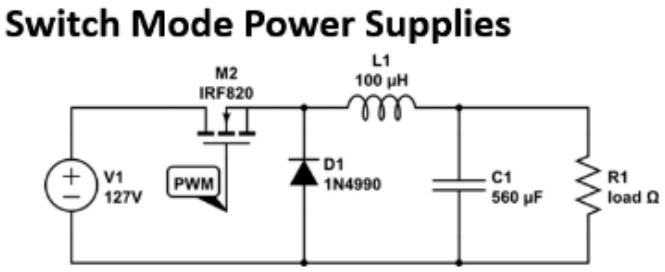
Poate regla tensiunea de ieșire chiar și atunci când tensiunea de alimentare e apropiată de cea de ieșire
• Zgomotul de comutare lipsește în cazul lor
• De asemenea disipa căldură în procesul de stabilizare a tensiunii de ieșire



Surse in comutatie

Transfer de putere de la o sursă DC sau AC la una DC
• Reglarea tensiunii prin raportul dintre timpul de pornire și oprire (duty cycle)
• Eficiență mai ridicată decât regulatoarele de tensiune liniare

• Mai complexe și mai scump



Stabilizator de tensiune

În tehnologia CMOS: P ~ V_dd^2 · f
Ce tranziții avem când în cod avem while(1); ?
• Încă se execută instrucțiuni, PC se modifică etc.
• Toate timerele active numără
• ADC-ul poate să fie activ
• Seriala poate transmite/recepționa
• Circuitele de input GPIO sunt active
• Power management - procedeul prin care restricționăm ce poate face procesorul

ATMEGA

Mod	Domenii active	Wake-up	Consum tipic (VCC = 3 V, 25 °C)
Active	clkCPU, clkFLASH, clkI/O, clkADC	—	1.5 mA @ 4 MHz
Idle	clkI/O, clkFLASH	INT, TWI, Timer, WDT	0.25 mA @ 4 MHz
ADC Noise Red.	clkADC	ADC, INT0/1	0.25 mA
Power-down	niciun ceas intern	WDT, INT0/1, TWI	40 µA
Power-save	clkASY (Timer 2)	Timer 2, WDT, INT	44 µA
Standby / Ext.	idem P-save + oscilator pornit	—	~60 µA

PICO

Mod RP2350	Ceasuri / domenii active	Regulator	Wake-up	Consum tipic[1]
RUN	SYS-clk (12-133 MHz), periferice selectate	VREG = 0.95–1.1 V	orice IRQ	25 mA @ 133 MHz
SLEEP (WFI)	SYS-clk la frecvenţă redusă (≤12 MHz)	VREG neschimbat	toate IRQ	0.5–1 mA
DORMANT	toate PLL/XOSC/ROSC OPRITE, doar LPOSC 20 kHz	VREG în LP 0.75 V	GPIO / RTC / WDT	5–15 µA
SWCORE_PD	core & SRAM parţial off, VREG 0.55 V	0.55 V	POR, BOD	<2 µA
RESET / OFF	—	—	POR	<1 µA

Clock gating în registre `CLK_*_CTRL` (similar PRR).

POWMAN / PSM controlează tranziţia în DORMANT / SWCORE_PD.

Regulator intern programabil `VREG.VSEL` (0.55–1.30 V).

LPOSC (≈20 kHz) pentru trezire temporizată fără PLL.

ATMEGA vs PICO

ATmega	RP2350
PRR (bit/ periferic)	`CLK_*_CTRL.ENABLE=0` (gating)
SMCR.SE + `sleep`	`cortex_m::asm::wfi()`
Power-save (Timer 2 doar)	DORMANT cu LPOSC sau Timer Alarms
BOD disable în sleep	BOD_CTRL, VREG LP entry
Current limit ≈ 40 µA P-down	5–10 µA DORMANT

Power-reduction = registru **PRR**, 1 bit/periferic.
Curent mediu: $I_{avg} = \sum I_i \cdot t_i / \sum t_i$.

Exemplu de calcul de curent – Pico 2 W

Fază	Timp	Curent
RUN 12 MHz (ADC+calc)	0.3 s	6 mA
DORMANT	59.7 s	10 µA = 0.01 mA

Iavg=606mA · 0.3 + 0.01mA · 59.7 ≈ 0.039mA

Baterie 1000 mAh → autonomie ≈ **25 700 h ≈ 2,9 ani**.

Optimizări practice pentru Pico 2 W

1. **Frecvenţă & Vcore scaling** – reduce SYS clk la 4–12 MHz și VREG 0.95 V.
2. **PLL & USB off** – mare parte din curentul activ.
3. **Detaşare periferice** – `spi.disable()`, `usb_phy.pd = 1`.
4. **LPOSC only** – foloseşte `embassy_time::Alarm` pentru trezire (RTC).
5. **Autopower transceiver/senzor** – MOSFET high-side controlat din GPIO.

Laboratoare

Intro to Rust

Variable types

let a = 5	immutable
let mut a = 5	mutable
const A: u32 = 5	immutable, value known at compile time

Data types

Length	Signed	Unsigned	C Equivalent
8-bit	i8	u8	char / unsigned char
16-bit	i16	u16	short / unsigned short
32-bit	i32	u32	int / unsigned int
64-bit	i64	u64	long long / unsigned long long
128-bit	i128	u128	N/A
arch	isize	usize	intptr_t / uintptr_t

Length	Floating Point	C Equivalent
32-bit	f32	float
64-bit	f64	double
128-bit	f128	N/A

Boolean

```
let t = true;
let f: bool = false;
```

Character

```
let c = 'z';
```

Structures

```
struct User {
    active: bool,
    username: String,
    email: String,
    sign_in_count: u64,
}
```

Structure Implemetation(Classes)

```
impl User {
    // called with User::new()
    fn new() -> User {
        // ...
    }
    // called user.is_active()
    fn is_active (&self) -> bool {
        return self.active;
    }
}
```

You can print structures that have #[derive(Debug)] with {?}

Tuple structuires

```
struct Color(i32, i32, i32);
struct Device(String, u8);
```

Enums

```
enum IpAddrKind {
    V4,
    V6,
}
```

Option enum

```
enum Option<T> {
    None,
    Some(T),
}
```

Match

```
match coin {
        Coin::Penny => 1,
        Coin::Nickel => 5,
        Coin::Dime => 10,
        Coin::Quarter => 25,
    }
```

Result Enum

```
enum Result<T,E> {
    Ok(T),
    Err(E),
}
```

The ? operator propagates an error to the caller func

Functions

```
fn another_function(x: i32) {
    println!("The value of x is: {x}");
}
```

Ownership

Ownership Rules in Rust

1. Each value has a single owner

Only one variable at a time can "own" a value. When ownership is transferred, the original owner can no longer use the value.

2. A value is dropped when its owner goes out of scope

Rust automatically frees memory by calling drop when the owner of a value exits its scope.

3. No value can have more than one owner at a time

This prevents multiple owners from modifying memory simultaneously, ensuring memory safety without a garbage collector.

Ownership and Functions

- Passing a value to a function moves ownership unless the value implements the Copy trait.
- To return ownership, functions can pass values back.
- Example: Strings (which do not implement Copy) are moved; primitive types like i32 (which do) are copied.

Borrowing and References

- Immutable reference: Allows reading without taking ownership (&T).
- Mutable reference: Allows modification but only one mutable reference is allowed at a time (&mut T).
- You cannot have a mutable and immutable reference to the same value at the same time.

Rules for References

- Only one mutable reference or multiple immutable references at a time.
- References must always be valid — they must not outlive the data they refer to.

The Copy Trait

- Types like i32, bool, and f64 implement Copy because they're small and safe to duplicate.
- Types like String or Vec<T> do not implement Copy due to heap allocation and pointer ownership.

Debug

Cross compile with: cargo build --target thumbv8m.main-none-eabihf

Sections

Type	Description
TEXT	Contains the binary code the processor executes
DATA	Contains data used by the firmware, usually variables
BSS	Contains uninitialized or zero-initialized data, usually global variables
DEBUG	Contains debug information used by the debugger software

Section	Fixed Offset	Summary Description
`.vector_table`	0x10000000	Holds initial stack pointer and interrupt vectors for system startup and interrupts.
`.start_block`	0x10000114	Stores boot metadata used by the bootloader for validation and execution.
`.text`	N/A	Contains executable code, usually in flash memory.
`.rodata`	N/A	Stores read-only data like constants and strings.
`.data`	N/A	Holds initialized global/static variables copied to RAM at startup.
`.bss`	N/A	Contains uninitialized or zero-initialized variables in RAM.
`.bi_entries`	N/A	Boot info used by the ROM bootloader and firmware updater.
`.end_block`	N/A	Marks firmware end for size checks and memory protection.

Pico2 no longer has .boot2

Message	Priority	Short Description
`trace!`	0	Detailed tracing for deep debugging.
`debug!`	1	Debug messages not shown in normal use.
`info!`	2	General informational messages.
`warn!`	3	Warnings for recoverable issues.
`error!`	4	Critical errors; firmware can't continue.

GPIO

Initializare periferice

```
let peripherals = embassy_rp::init(Default::default());
```

Initializare pin

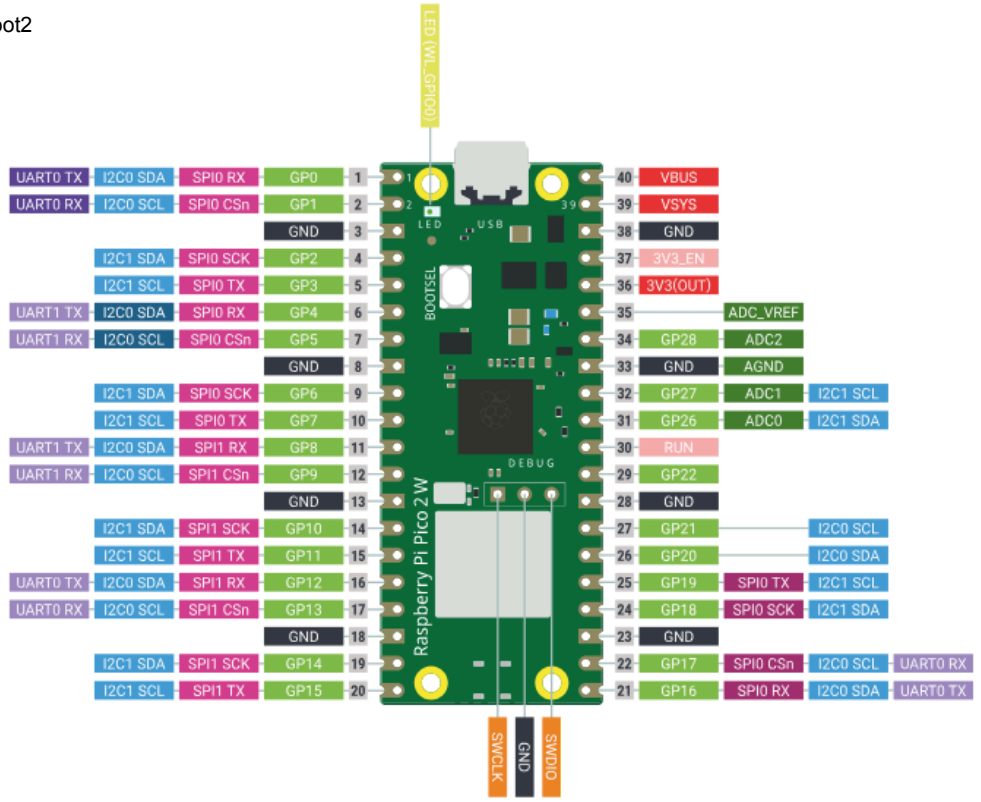
```
let mut pin = Output::new(peripherals.PIN_n, Level::Low);
let pin = Input::new(peripherals.PIN_n, Pull::Up);
```

Functii:

- output
  - pin.set_high/low();
- input
  - pin.is_high/low(); - verificare instantanee
  - pin.wait_for_(high/low/any_edge/falling_edge/rising_edge).await

Timer:

Timer::after_secs(1).AWAIT



PWM & ADC

Setup PWM

```
// PWM config
use embassy_rp::pwm::Config as ConfigPwm;

// Create config for PWM slice
let mut config: ConfigPwm = Default::default();
// Set top value (value at which PWM counter will reset)
config.top = 0x9088; // in HEX, equals 37000 in decimal
// Set compare value (counter value at which the
// PWM signal will change from 1 to 0)
config.compare_a = config.top / 2;


// Create a PWM driver for pin 3
let mut pwm = Pwm::new_output_b( // Output B
    peripherals.PWM_SLICE1, // Channel 1
    peripherals.PIN_3,   // Pin 3 (modify this as needed)
    config.clone()
);
```

Change PWM configuration

```
config.compare_b += 100; // modified value of `compare_b`
pwm.set_config(&config); // set the new configuration for
PWM
```

Using a servo with PWM

Servos need a frequency of 50 HZ => we use a divider

$$top = \left(\frac{f_{clock}}{f_{pwm} \times divider}\right) - 1$$

$$f_{clock} = 150\,MHz$$
$$f_{PWM} = 50\,Hz$$
$$divider = 64$$

$$top = \left(\frac{150\,000\,000}{50 \times 64}\right) - 1 = 46\,874$$

0 degrees => 0.5 ms pulse
180 degrees => 2.5 ms pulse

$$compare = \left(\frac{pulse_{width} \times top}{T}\right)$$

```
use embassy_rp::adc::Config as ConfigAdc; // ADC config

// ---- fn main() ----
let mut adc = Adc::new(peripherals.ADC, Irqs,
ConfigAdc::default());
let level = adc.read(&mut adc_pin).await.unwrap();
```

Asynchronous Development

Example async function

```
#[embassy_executor::task]
async fn button_pressed(mut led: Output<'static>, mut
button: Input<'static>) {
    loop {
        info!("waiting for button press");
        button.wait_for_falling_edge().await;
        led.toggle();
    }
}
main:
    let button = Input::new(peripherals.PIN_X,
Pull::None);
    let led2 = Output::new(peripherals.PIN_X,
Level::Low);

    spawner.spawn(button_pressed(led2, button)).unwrap();
```

Select function

Returns the first future that happens

```
let select = select(button.wait_for_falling_edge(),
Timer::after_secs(5)).await;
match select {
    First(res1) => {
        // handle case for button press
    },
    Second(res2) => {
        // handle case for alarm firing
    }
}
```

Join function

Waits for both futures to happen

```
let (res1, res2) = join(button.wait_for_falling_edge(),
Timer::after_secs(5)).await;
```

Channels

```
static CHANNEL: Channel<ThreadModeRawMutex, bool, 64> =
Channel::new();
func1:
        CHANNEL.send(true).await;
func2:
        let value = CHANNEL.receive().await;
```

Signal

```
static SIG: Signal<CriticalSectionRawMutex, ()> =
Signal::new();
func1:
        SIG.signal(());
func2:
        SIG.wait().await;
```

PubSubChannel

```
static PUB: PubSubChannel<CriticalSectionRawMutex,
&'static str, 4, 2> = PubSubChannel::new();
publisher:
        PUB.publisher().publish("Hello").await;
sub1:
        let mut sub = PUB.subscriber().unwrap();
sub2:
        let mut sub = PUB.subscriber().unwrap();
```

GPIO	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
PWM Channel	0A	0B	1A	1B	2A	2B	3A	3B	4A	4B	5A	5B	6A	6B	7A	7B
GPIO	16	17	18	19	20	21	22	23	24	25	26	27	28	29		
PWM Channel	0A	0B	1A	1B	2A	2B	3A	3B	4A	4B	5A	5B	6A	6B		

Serial Peripheral Interface

Setting up SPI

```
// spi config
let mut config = spi::Config::default();
config.frequency = 1_000_000;
config.phase = spi::Phase::CaptureOnFirstTransition;
config.polarity = spi::Polarity::IdleLow;

let miso = p.PIN_X;
let mosi = p.PIN_Y;
let clk = p.PIN_Z;

let mut spi = Spi::new(p.SPI0, clk, mosi, miso,
p.DMA_CH0, p.DMA_CH1, config);

let mut cs = Output::new(p.PIN_N, Level::High);

// transferring data through spi
cs.set_low();
let tx_buf = [1_u8, 2, 3, 4, 5, 6];
let mut rx_buf = [0_u8; 6]; // dummy values, will be
replaced by received data
spi.transfer(&mut rx_buf, &tx_buf).await;
cs.set_high();
```

MPU-6500 - gyroscope and accelerometer
Reading from the sensor

```
cs.set_low();
let tx_buf = [(1 << 7) | REG_ADDR, 0x00]; // first value
of buffer is the control byte, second is a *don't care*
value
let mut rx_buf = [0u8; 2]; // initial values that will be
replaced by the received bytes
spi.transfer(&mut rx_buf, &tx_buf).await.unwrap();
cs.set_high();
let register_value = rx_buf[1]; // the second byte in the
buffer will be the received register value
```

Writing to the sensor

```
cs.set_low();
let tx_buf = [!(1 << 7) & REG_ADDR, value_to_write]; //
value_to_write is to be replaced with the 8-bit value
that we want to write to this register
let mut rx_buf = [0u8; 2]; // we are not expecting any
relevant information to be received, but we still need to
receive dummy values anyway
spi.transfer(&mut rx_buf, &tx_buf).await;
```

Inter-Integrated Circuit(I2C)

```
// setup
let sda = peripherals.PIN_X;
let scl = peripherals.PIN_Y;

/// I2C
let mut i2c = I2c::new_async(peripherals.I2CX, scl, sda,
Irqs, I2cConfig::default());

// reading
const TARGET_ADDR: u16 = 0x44;
let mut rx_buf = [0x00u8; 2];
i2c.read(TARGET_ADDR, &mut rx_buf).await.unwrap();

// writing
const TARGET_ADDR: u16 = 0x44;
let tx_buf = [0x01, 0x05];
i2c.write(TARGET_ADDR, &tx_buf).await.unwrap();
```

BMP280
Digital pressure and temperature sensor

Reading from sensor

```
// The tx buffer contains the address of the register we
want to read
let tx_buf = [REG_ADDR];
// the rx buffer will contain the value of the requested
register
// after the transfer is complete.
let mut rx_buf = [0x00u8];
// The function's arguments are the I2C address of the
BMP280 and the two buffers
i2c.write_read(BMP280_ADDR, &tx_buf, &mut
rx_buf).await.unwrap();
```

Writing to a register

```
let tx_buf = [REG_A, 0x00];
i2c.write(BMP280_ADDR, &tx_buf).await.unwrap();

let tx_buf = [REG_A, 0x00, REG_B, 0x00];
i2c.write(BMP280_ADDR, &tx_buf).await.unwrap();
```

EEPROM = Electrically Erasable Programmable Read-Only Memory - memory that persists even when the device is powered off

WIFI

IPv4

An IPv4 address looks like this: `192.168.0.1/24`.

Each number separated by a dot is represented on 8 bits (ranging from 0 to 255). The number after the slash is the *network mask*, which will be described shortly.

`192.168.0.1` = `11000000.10101000.00000000.00000001`

An IP address is made of two parts:

- **network prefix**: this portion defines the network to which the device belongs
- **host identifier**: this portion uniquely identifies the device in the network

The *network mask* defines the number of bits that are used to represent the network prefix. A smaller mask allows more bits to be used for host identifiers, enabling a larger number of unique devices within the network.

For example, for the above address `192.168.0.1/24`, we know that the network prefix is 24 bits long. Therefore, the first 24 bits give us the network prefix and the last 8 bits give us the host identifier. The mask in this case would look like this: `255.255.255.0`.

Network masks

In binary, `255.255.255.0` is `11111111.11111111.11111111.00000000`.

The first 24 bits are `1`, the last 8 bits are `0`.

Any device that wants to communicate in this local network (or *subnet*) must have an address ranging from `192.168.0.1` to `192.168.0.254`.

Wi-Fi on the Raspberry Pi Pico 2 W

Apart from the RP2350 microcontroller, the Raspberry Pi Pico 2W board also has a separate, on-board wireless interface. It is a 2.4 GHz Infineon chip, *CYW43439*. This chip deals only with Wi-Fi (and Bluetooth) communication, and it is connected to the RP2350 via SPI.

Code

```
// === Firmware loading ===
let fw = include_bytes!("../cyw43-firmware/43439A0.bin");
let clm =
include_bytes!("../cyw43-firmware/43439A0_clm.bin");

// === SPI over PIO setup ===
let cs = Output::new(p.PIN_25, Level::High);
let mut pio = Pio::new(p.PIO0, Irqs);
let spi = PioSpi::new(&mut pio.common, pio.sm0, pio.irq0,
cs, p.PIN_24, p.PIN_29, p.DMA_CH0);

// === Interrupts for PIO ===
bind_interrupts!(struct Irqs {
    PIO0_IRQ_0 => InterruptHandler<PIO0>;
});

// === Start CYW43 Wi-Fi driver ===
let pwr = Output::new(p.PIN_23, Level::Low);
static STATE: StaticCell<cyw43::State> =
StaticCell::new();
let state = STATE.init(cyw43::State::new());
let (net_device, mut control, runner) = cyw43::new(state,
pwr, spi, fw).await;
spawner.spawn(wifi_task(runner)).unwrap();
control.init(clm).await;
control.set_power_management(cyw43::PowerManagementMode::
None).await;

#[embassy_executor::task]
async fn wifi_task(runner: cyw43::Runner<'static,
Output<'static>, PioSpi<'static, PIO0, 0, DMA_CH0>>) -> !
{
    runner.run().await
}

// === Network config ===
// Option 1: Static IP
let config =
embassy_net::Config::ipv4_static(embassy_net::StaticConfi
gV4 {
   address: Ipv4Cidr::new(Ipv4Address::new(192, 168, 0,
2), 24),
   dns_servers: Vec::new(),
   gateway: Some(Ipv4Address::new(192, 168, 0, 1)),
});
// Option 2: DHCP
let config = Config::dhcpv4(Default::default());

// === Network stack init ===
static RESOURCES: StaticCell<StackResources<2>> =
StaticCell::new();
let (stack, runner) = embassy_net::new(net_device,
config, RESOURCES.init(StackResources::new()),
0x0123_4567_89ab_cdef);
spawner.spawn(net_task(stack));

#[embassy_executor::task]
async fn net_task(mut runner:
embassy_net::Runner<'static, cyw43::NetDriver<'static>>)
-> ! {
    runner.run().await
}

// === Scan for nearby Wi-Fi networks ===
let mut scanner = control.scan(Default::default()).await;
while let Some(bss) = scanner.next().await {
    if let Ok(ssid_str) = core::str::from_utf8(&bss.ssid)
{
        info!("Scanned {}", ssid_str);
    }
}
```

```
// === Connect to WPA2 Wi-Fi ===
const WIFI_NETWORK: &str = "my_ssid";
const WIFI_PASSWORD: &str = "password";
loop {
    match control.join(WIFI_NETWORK,
JoinOptions::new(WIFI_PASSWORD.as_bytes())).await {
        Ok(_) => break,
        Err(err) => info!("join failed with status={}",
err.status),
    }
}

// Wait for network stack to be configured (via DHCP or
static IP)
info!("waiting for IP config...");
while !stack.is_config_up() {
    Timer::after_millis(100).await;
}
info!("Network is ready!");

// === Open network ===
control.join(WIFI_NETWORK,
JoinOptions::new_open()).await;

// === Start Access Point ===
control.start_ap_open("Pico AP", 5).await;
// open
control.start_ap_wpa2("Pico AP", "WeloveRust", 5).await;
// WPA2 protected

// === TCP server ===
let mut rx_buffer = [0; 4096];
let mut tx_buffer = [0; 4096];
let mut socket = TcpSocket::new(stack, &mut rx_buffer,
&mut tx_buffer);
socket.set_timeout(Some(Duration::from_secs(10)));
info!("Listening on TCP:1234...");
if let Err(e) = socket.accept(1234).await {
    warn!("accept error: {:?}", e);
}
info!("Received connection from {:?}",
socket.remote_endpoint());

// TCP read/write loop
let mut buf = [0; 4096];
loop {
    let n = match socket.read(&mut buf).await {
        Ok(0) => { warn!("read EOF"); break; }
        Ok(n) => n,
        Err(e) => { warn!("read error: {:?}", e); break;
}
    };
    info!("received: {}",
core::str::from_utf8(&buf[..n]).unwrap());
    if let Err(e) = socket.write_all(&buf[..n]).await {
        warn!("write error: {:?}", e);
        break;
    }
}

// === TCP client ===
let endpoint = IpEndpoint::new(IpAddress::v4(1,2,3,5),
1234);
if let Err(e) = socket.connect(endpoint).await {
    warn!("connect error: {:?}", e);
}

// === UDP server ===
let mut rx_buffer = [0; 4096];
let mut rx_meta = [PacketMetadata::EMPTY; 3];
let mut tx_buffer = [0; 4096];
let mut tx_meta = [PacketMetadata::EMPTY; 3];
let mut socket = UdpSocket::new(stack, &mut rx_meta, &mut
rx_buffer, &mut tx_meta, &mut tx_buffer);
```

```
// Bind to port 1234
socket.bind(1234).unwrap();

// Receive UDP packets
let mut buf = [0u8; 4096];
loop {
    match socket.recv_from(&mut buf).await {
        Ok((n, endpoint)) => {
            info!("Received from {:?}: {}", endpoint,
core::str::from_utf8(&buf[..n]).unwrap());
        },
        Err(_) => {
            info!("An error occurred when receiving the
packet!");
        }
    }
}

// Send UDP packet to specific endpoint
let buffer = b"hello\n";
let target = IpEndpoint::new(IpAddress::v4(192, 168, 100,
45), 1234);
match socket.send_to(&buffer[..], target).await {
    Ok(()) => info!("sent"),
    Err(e) => warn!("send error: {:?}", e),
}

// === Lab helper macros ===
let (net_device, mut control) = init_wifi!(&spawner,
peripherals).await;
let stack = init_network_stack(&spawner, net_device,
&RESOURCES, config);
```

FAST PWM

Enunț: Presupunem că avem un semnal PWM generat de Timer 1 configurat în mod Fast PWM 8-bit, non-inverting, cu prescaler 64, cu OCRIA stat la 64, iar frecvența de ceas a UC este de 16MHz. Calculați durata de timp pentru care semnalul este HIGH în cadrul unui ciclu (HIGH - LOW).

Soluție:

În modul Fast PWM 8-bit:
- Contorul numără de la 0 la 255 (TOP=255)
- Semnalul este HIGH de la 0 până la OCRIA (64)
- Formula pentru calculul frecvenței PWM este: f_PWM = f_clk / (N * 256), unde N este prescalerul

Calculul duratei pentru care semnalul este HIGH:

1. Mai întâi calculăm perioada unui tick:
   T_tick = prescaler / f_clk = 64 / 16MHz = 64 / 16.000.000 = 0,000004 s = 4 µs

2. În modul non-inverting, semnalul este HIGH de la 0 până la OCRIA (64):
   Durata HIGH = OCRIA * T_tick = 64 * 4 µs = 256 µs

Deci, semnalul este HIGH pentru 256 µs în cadrul unui ciclu PWM.

Performanta CPU

Enunț: Avem două CPU-uri cu următoarele caracteristici, având același set de instrucțiuni:
- CPU 1: Frecvența ceasului = 2GHz, CPI (cicluri medii pe instrucțiune) = 1,5
- CPU 2: Frecvența ceasului = 1600MHz, CPI = 1
Care dintre cele două este mai performant?

Soluție:

Pentru a compara performanța celor două CPU-uri, voi calcula timpul mediu de execuție per instrucțiune:

Pentru CPU 1:
- Frecvența ceasului = 2 GHz = 2 * 10^9 Hz
- CPI = 1,5
- Timp de execuție per instrucțiune = CPI / Frecvența ceasului = 1,5 / (2 * 10^9) = 0,75 * 10^-9 s = 0,75 ns

Pentru CPU 2:
- Frecvența ceasului = 1600 MHz = 1,6 * 10^9 Hz
- CPI = 1
- Timp de execuție per instrucțiune = CPI / Frecvența ceasului = 1 / (1,6 * 10^9) = 0,625 * 10^-9 s = 0,625 ns

Comparând cele două valori:
- CPU 1: 0,75 ns per instrucțiune
- CPU 2: 0,625 ns per instrucțiune

Cum CPU 2 execută o instrucțiune în mai puțin timp (0,625 ns < 0,75 ns), CPU 2 este mai performant decât CPU 1, chiar dacă are o frecvență de ceas mai mică. Acest lucru se datorează faptului că CPU 2 necesită mai puține cicluri de ceas pentru a executa o instrucțiune (CPI=1 față de CPI=1,5).

Număr senzori

Enunț: Aveți de implementat un sistem ce necesită achiziția de date la frecvența mică de la un număr mare de senzori digitali. Aveți la dispoziție un RP2350 și libertatea de a alege tipul de senzori și interfața de comunicare cu aceștia. Ce interfață de comunicare alegeți? Care este numărul maxim de senzori pe care îi puteți conecta?

Soluție:

Pentru implementarea unui sistem cu RP2350 care necesită conectarea unui număr mare de senzori digitali cu frecvență mică de achiziție, aleg interfața I²C (Inter-Integrated Circuit).

Justificare pentru alegerea I²C cu RP2350:

1. Arhitectura RP2350: Acest microcontroler are două controlere I²C hardware dedicate care suportă comunicare la viteze de până la 1 Mbps (Fast Mode Plus). Fiind bazat pe arhitectura ARM Cortex-M0+, oferă performanță excelentă pentru procesarea datelor de la numeroși senzori.

2. Eficiență în cablare: I²C folosește doar două linii (SDA și SCL) pentru comunicare cu multiple dispozitive, ceea ce reduce semnificativ complexitatea circuitului și costul implementării.

3. Adecvare pentru frecvențe mici: Cum sistemul necesită achiziție la frecvență mică, I²C este perfect adecvat - este optimizat pentru comunicații periodice, nu continue, cu overhead redus.

4. Suport pentru Rust: RP2350 are suport excelent în ecosistemul Rust embedded prin biblioteci precum `rp-hal` și `embedded-hal`, care oferă abstracțiuni sigure pentru I²C, beneficiind de verificările de siguranță la compilare specifice Rust.

5. Avantaj PIO: RP2350 include blocuri PIO (Programmable I/O) care permit implementarea flexibilă a protocoalelor de comunicație, inclusiv I²C suplimentar, pe aproape orice pini GPIO disponibili.

Numărul maxim de senzori conectabili:

Cu RP2350 și interfața I²C:

1. Teoretic: Standardul I²C permite 7 biți pentru adresare, rezultând 2^7 = 128 adrese posibile. Luând în considerare adresele rezervate, aproximativ 112-120 de dispozitive pot fi adresate direct.

2. Extinderea capacității: Folosind multiplexoare I²C dedicate (cum ar fi TCA9548A cu 8 canale), putem extinde sistemul. Cu un singur multiplexor putem conecta: 8 canale × ~110 adrese = ~880 senzori.

3. Implementare multi-bus: Folosind cele două interfețe I²C hardware ale RP2350 și implementarea suplimentară prin PIO, putem crea 3-4 magistrale I²C paralele, crescând și mai mult numărul de senzori conectabili.

4. Limitări practice: În practică, numărul este limitat de:
   - Capacitanța totală a magistralei (sub 400pF)
   - Timpul necesar pentru interogarea periodică a tuturor senzorilor
   - Managementul energetic pentru numărul mare de senzori

Concluzie: Cu RP2350 și interfața I²C, pot fi conectați teoretic până la ~880 senzori folosind multiplexoare, iar practic în jur de 200-300 de senzori, luând în considerare limitările fizice și cerințele de performanță pentru achiziția de date la frecvență mică.
Exercițiul 7 [6p] - Sistem de monitorizare a poluării bazat pe RP2350

Senzor poluare

Cerințele principale sunt:
- Implementarea unui senzor de poluare pentru monitorizarea gradului de poluare de birou
- Sistemul nu va fi alimentat de la priză și nu necesită schimbarea frecventă a bateriei
- Avem disponibilă o baterie de 6V, 400mAh
- Eșantionare la fiecare 60 secunde
- Trebuie calculată durata maximă de funcționare
- Implementare hardware-software optimizată energetic

1. Schema bloc hardware

2. Electronica din spatele deciziilor - Analiză detaliată

A. Alegerea microcontrolerului RP2350

Justificare electronică:
- Nucleul ARM Cortex-M0+ este optimizat pentru consum redus (0.9 DMIPS/MHz) și oferă performanțe suficiente pentru procesarea datelor de la senzorii de poluare
- Tensiune de operare: 1.8V-3.6V, perfectă pentru sisteme alimentate cu baterii
- Moduri de sleep avansate:
  - Sleep mode: 0.8mA la 3.3V
  - Dormant mode: ~0.1mA
  - Shutdown mode: <0.003mA (3µA)
- Periferice cu consum redus: ADC cu auto-scan, ce poate face conversii fără a trezi CPU-ul complet

B. Sistemul de alimentare

Convertor DC-DC Buck (Step-Down):
- Componentă recomandată: TPS62840 (Texas Instruments) sau MCP1700 (Microchip)
- Justificare electronică:
  - Eficiență de conversie >90% la curenți mici (comparat cu 40-50% pentru regulatoarele liniare)
  - Quiescent current (curent de repaus) extrem de mic: ~60nA
  - Tensiune de ieșire: 3.3V stabilizată pentru RP2350
  - Capacitate de operare în mod "power save" la sarcini mici

Calcule de eficiență energetică:
- Bateria de 6V trebuie convertită la 3.3V pentru alimentarea microcontrolerului
- Energia disponibilă în baterie: 6V × 0.4Ah = 2.4Wh
- Cu o eficiență de conversie de 90%: 2.16Wh disponibili pentru sistem

C. Senzori de poluare

Alegere: BME680 (Bosch) - senzor multi-parametric pentru calitatea aerului
- Justificare electronică:
  - Interfață I²C cu consum redus
  - Măsoară: VOC (compuși organici volatili), temperatură, umiditate, presiune
  - Consum în mod activ: ~3.7mA pentru 1 măsurare
  - Consum în mod sleep: ~0.15µA
  - Timpul de măsurare: ~160ms

Alternativă: SGP40 (Sensirion) pentru măsurare specifică a VOC
- Consum și mai redus: ~2.6mA în mod activ, ~0.07µA în standby

D. Managementul energiei

Strategia de economisire a energiei:

1. Power gating: Alimentarea senzorului este controlată printr-un MOSFET de tip N cu un RDSon mic (ex: DMG2305UX)
   - Justificare electronică: Când senzorul nu este utilizat, alimentarea este complet tăiată, eliminând curentul de standby
   - Economie: ~0.15µA × 99% din timp = ~0.1485µA economisiți per ciclu

2. Clock gating și power domains:
   - RP2350 permite oprirea selectivă a domeniilor de alimentare și a ceasurilor pentru periferiale
   - Justificare electronică: Oprirea clk_peri și clk_adc când nu sunt utilizate reduce consumul cu ~30-40%

3. Duty cycling optimizat:
   - Timp activ: ~200ms (160ms măsurare + 40ms procesare)
   - Timp sleep: 59.8s
   - Raport duty cycle: 0.33%

E. Software și firmware

Strategii de optimizare software:

1. Implementare bazată pe întreruperi:
   - Utilizarea RTC (Real-Time Clock) pentru trezirea sistemului la fiecare 60 secunde
   - Justificare electronică: Evită polling-ul și permite CPU să rămână în sleep maximum de timp

2. Secvența de execuție optimizată:

```
    loop {
        // Activează alimentarea senzorului (power gate)
        enable_sensor_power();

        // Așteaptă stabilizarea senzorului (10ms)
        delay_ms(10);

        // Configurează și inițiază măsurarea
        setup_sensor();
        start_measurement();

        // Pune MCU în sleep light în timpul măsurării
        enter_light_sleep();  // CPU oprit, periferice
active

        // Citește și procesează datele
        let readings = read_sensor_data();
        process_and_store(readings);

        // Dezactivează alimentarea senzorului
        disable_sensor_power();

        // Intră în deep sleep pentru restul perioadei de
60s
        enter_deep_sleep_until_rtc_alarm();
    }
```

3. Optimizări la nivel de compilator:
   - Utilizarea flag-urilor de optimizare `-C opt-level=z` pentru minimizarea dimensiunii codului
   - LTO (Link Time Optimization) pentru eliminarea funcțiilor neutilizate

3. Calculul duratei maxime de funcționare

Consum de energie per ciclu de 60 secunde:

1. Faza activă (200ms):
   - RP2350 în mod activ: ~5mA × 3.3V × 200ms = 3.3mJ
   - Senzor BME680 activ: ~3.7mA × 3.3V × 160ms = 1.95mJ
   - Circuit auxiliar (convertor, etc.): ~0.1mA × 3.3V × 200ms = 0.066mJ
   - Total fază activă: ~5.32mJ

2. Faza sleep (59.8s):
   - RP2350 în dormant mode: ~0.1mA × 3.3V × 59.8s = 19.73mJ
   - hirtor DC-DC (quiescent): ~0.00006mA × 3.3V × 59.8s = 0.01mJ

- Total fază sleep: ~19.74mJ

3. Consum total per ciclu: 5.32mJ + 19.74mJ = 25.06mJ

Calcul durată de funcționare:
- Energie disponibilă: 2.16Wh = 7,776J
- Consum per ciclu: 25.06mJ
- Număr total de cicluri: 7,776J ÷ 0.02506J = ~310,295 cicluri
- Durată totală: 310,295 cicluri × 60 secunde = 18,617,685 secunde ≈ 216 zile

4. Optimizări electronice suplimentare

1. Capacitate de filtrare optimizată:
   - Condensatoare ceramice de 1μF pentru decuplarea locală a microcontrolerului
   - Justificare electronică: Condensatoarele ceramice au curent de scurgere (leakage) mai mic comparativ cu condensatoarele electrolitice sau tantalum

2. Circuite de protecție:
   - Diodă Schottky cu curent de scurgere mic pentru protecția la polaritate inversă
   - Justificare electronică: Curent de scurgere <1μA, comparativ cu >10μA pentru diodele standard

3. Senzor de tensiune baterie:
   - Divizor rezistiv conectat la ADC, activat doar periodic (1 dată/zi)
   - Justificare electronică: Rezistențe de valoare mare (>1MΩ) pentru minimizarea curentului prin divizor

4. PCB layout optimizat:
   - Separarea planurilor de masă pentru circuite digitale și analogice
   - Justificare electronică: Reducerea zgomotului și a curenților de scurgere între domenii

5. Comunicare și afișare opțională

Pentru comunicare, vom implementa un modul Bluetooth Low Energy (BLE) care va fi activat rar (la cerere sau o dată pe zi):

- Modul recomandat: DA14531 (Dialog Semiconductor)
- Justificare electronică:
  - Consum extrem de redus: <1μA în deep sleep
  - Consum în transmisie: ~4.5mA pentru ~5ms per pachet
  - Sistem de operare real-time optimizat pentru BLE

Concluzie

Această implementare hardware-software pentru un sistem de monitorizare a poluării bazat pe RP2350 poate funcționa aproximativ **216 zile** cu o singură baterie de 6V și 400mAh, respectând toate cerințele impuse. Prin optimizările electronice propuse și managementul eficient al energiei, sistemul va funcționa timp îndelungat fără necesitatea înlocuirii bateriei, fiind ideal pentru monitorizarea de lungă durată a calității aerului într-un birou.

Trotinete

6.1 Schema bloc

6.2 Pseudocod - implementare conceptuală

```
// Structura pentru datele IMU
struct ImuData {
    accel_x, accel_y, accel_z  // Accelerometru pe 3 axe
    gyro_x, gyro_y, gyro_z     // Giroscop pe 3 axe
    timestamp                  // Marcaj temporal
}

// Buffer circular pentru memorarea eșantioanelor
buffer = CircularBuffer(capacitate=100)  // Ultimele 100 de eșantioane

// Semnal pentru comunicare între task-uri
signal = Signal()

// TASK 1: Eșantionare IMU la frecvență înaltă (prioritate medie)
functia imu_sampling_task():
    // Configurare inițială a senzorului IMU
    configureaza_imu()

    // Buclă infinită de eșantionare
    cat timp true:
        // Marcare timp de început pentru sincronizare precisă
        timp_start = timp_curent()

        // Citire date de la senzorul IMU
        date_imu = citeste_imu()

        // Adaugă eșantionul nou în buffer
        buffer.adauga(date_imu)

        // Așteaptă până la următoarea perioadă de eșantionare (10kHz)
        // Pentru a menține frecvența constantă de 10kHz (100μs)
        timp_scurs = timp_curent() - timp_start
        daca timp_scurs < 100μs:
            asteapta(100μs - timp_scurs)

// TASK 2: Detecție evenimente (prioritate medie)
functia event_detection_task():
    // Fereastra de analiză (5ms = 50 eșantioane la 10kHz)
    fereastra_analiza = []

    cat timp true:
        // Așteaptă până avem un nou eșantion disponibil
        daca buffer.disponibil():
            esantion = buffer.citeste_cel_mai_nou()

            // Actualizează fereastra glisantă
            adauga_la_fereastra(fereastra_analiza, esantion, max_size=50)

            // Analizează fereastra dacă este completă (5ms de date)
            daca fereastra_analiza.dimensiune() >= 50:
                // Apel funcție externă de detecție (specificată în enunț)
                daca event_detection(fereastra_analiza) == 1:
                    // Semnalează detectarea accidentului
                    signal.activeaza()

                    // Curăță fereastra pentru a evita detecții multiple
                    fereastra_analiza.curata()

// TASK 3: Control airbag (prioritate maximă)
```

```
functia airbag_control_task():
    cat timp true:
        // Așteaptă semnalul de accident detectat
        signal.asteapta()

        // Secțiune critică pentru timing precis - dezactivează întreruperile
        dezactiveaza_intreruperi()
        {
            // Generează impuls de 10µs pentru comandă airbag
            pin_airbag = HIGH
            intarziere_precisa(10µs)  // Delay precis bazat pe cicluri CPU
            pin_airbag = LOW
        }
        reactivare_intreruperi()


// Funcția principală
functia main():
    // Inițializare hardware
    configureaza_gpio()
    configureaza_spi()

    // Pornire task-uri concurente cu prioritățile adecvate
    porneste_task(imu_sampling_task, prioritate=1)
    porneste_task(event_detection_task, prioritate=1)
    porneste_task(airbag_control_task, prioritate=2)  // Prioritate maximă
```

6.3 Pseudocod pentru soluția alternativă - blocul "comandă airbag"

```
// VERSIUNEA ALTERNATIVĂ PENTRU BLOCUL COMANDĂ AIRBAG

// Configurare hardware
functia setup():
    // Configurare pin de intrare (pentru primirea semnalului)
    configureaza_pin_intrare(PIN_COMANDA, cu_intrerupere=true, muchie=RISING)

    // Configurare pin de ieșire (pentru controlul tranzistorului)
    configureaza_pin_iesire(PIN_TRANZISTOR, initial=LOW, viteza=MAXIMA)

    // Configurare întreruperi cu prioritate maximă
    configureaza_intrerupere(PIN_COMANDA, prioritate=MAXIMA)

// Rulare principală
functia main():
    // Inițializare sistem
    setup()

    // Buclă principală simplă - totul se face în rutina de întrerupere
    cat timp true:
        // Mod sleep pentru economie de energie
        intra_in_sleep()

// Rutina de tratare a întreruperii - se execută automat la detectarea muchiei
functia intrerupere_comanda():
    // Secțiune critică - dezactivează alte întreruperi
    dezactiveaza_intreruperi()

    // Generează puls de comandă pentru tranzistor
    PIN_TRANZISTOR = HIGH

    // Delay precis de 10µs folosind cicluri CPU
    // Pentru RP2350 la 133MHz: ~1333 cicluri = 10µs
    delay_cicluri(1333)


    // Închide comanda
    PIN_TRANZISTOR = LOW

    // Reactivează întreruperile
    activeaza_intreruperi()
```

Explicația conceptelor importante

1. Eșantionarea IMU la frecvență înaltă (10kHz)

- Senzorul IMU trebuie să citească date la o frecvență foarte mare pentru a detecta impacturile rapide
- Timing-ul precis este esențial - fiecare eșantion trebuie luat la exact 100µs după precedentul
- Această frecvență înaltă permite detectarea celor mai subtile schimbări în accelerație și orientare

2. Analiza pe fereastra glisantă de 5ms

- Fereastra de 5ms (50 de eșantioane) oferă suficiente date pentru a detecta un pattern de accident
- Abordarea cu fereastră glisantă asigură că nu se pierde niciun eveniment între analize
- Funcția event_detection analizează întreaga fereastră pentru a identifica semnăturile specifice impacturilor

3. Prioritizarea task-urilor

- Task-ul de control airbag are prioritate maximă pentru a asigura execuția imediată când este detectat un accident
- Task-urile de eșantionare și detecție au priorități mai mici dar suficiente pentru a asigura fluxul de date

4. Secțiunea critică pentru timing precis

- Pentru generarea pulsului de exact 10µs, întreruperile trebuie dezactivate
- Delay-ul este realizat prin numărarea precisă a ciclurilor CPU (nu prin funcții de delay obișnuite)
- Aceasta asigură că pulsul are exact durata cerută, indiferent de alte evenimente din sistem

5. Comunicarea între task-uri prin semnale

- Folosirea unui semnal simplu pentru notificarea task-ului de control airbag
- Această abordare are latență minimă și evită overhead-ul comunicațiilor complexe

6. Soluția alternativă bazată pe întreruperi

- Soluția alternativă elimină task-urile și folosește direct întreruperi
- Aceasta reduce și mai mult latența, ajungând la timpul minim teoretic (~400ns)
- Întreaga logică este executată în contextul întreruperii, ceea ce asigură timp de răspuns minim

Calcul estimativ al timpului de răspuns:

- ~12 cicluri pentru intrarea în rutina de întrerupere
- ~10 cicluri pentru procesarea inițială
- ~5 cicluri pentru setarea pin-ului de ieșire
- Total: ~27 cicluri = ~200ns la 133MHz